How a Proof Becomes a Program

A tour of two formal systems in which theorems, once proved, hand you a working program — and of how the proof, its logical strength, and its choice of representation each leave a mark on the program you get.

Ara Aslyan

Abstract

We present a mechanized framework for modified realizability over finite-type intuitionistic arithmetic, formalized in Lean 4, in which the extracted realizer is a term of the object language rather than an opaque function. The framework proves extraction sound and extracted realizers continuous, and extends the object theory by a *matched* proof/program pair — transfinite induction up to ε_0 together with a recursor along the induced ordinal order — so that theorems beyond arithmetical induction still yield running programs; case-study symbols enter as primitives evaluated by separately verified value layers.

Thirteen theorems are derived and extracted, among them Goodstein's theorem and the Kirby–Paris hydra theorem in a strategy-quantified form the one-sorted setting cannot state. The system makes proof structure, logical strength, and representation observable in the extracted programs: two proofs of one theorem extract to programs that return different witnesses; replacing a numerical encoding of hydra trees by a structural type turns a battle whose coded extract exhausts the evaluator into one that computes the published length, while an analogous change for ordinals reduces representation size without any measured change in running time; and type-2 realizers are continuous, with a discontinuous functional exhibited that no derivation extracts to. A translation into a functional target language is proved correct against a formal semantics of that target for the non-transfinite fragment. Every extracted program that runs is machine-checked to depend on no choice principle.

Prologue

In school we learn that a proof is something that convinces us a statement is true. A proof of "every even number greater than two is a sum of two primes" would settle the matter; it would not, on the face of it, *do* anything.

Computer scientists discovered something stronger. Sometimes a proof does not merely establish that an object exists — it contains, inside itself, a recipe for *building* that object. A proof that "every list can be sorted" can hide an actual sorting algorithm in its steps. When that happens, the proof is not just an argument; it is a program in disguise, and with the right lens you can read the program straight off the proof. Proof assistants — software that checks proofs down to the last symbol — let us apply that lens *mechanically*, turning a verified proof in a formal proof language built for extraction into runnable code with no human cleverness in between.[1]

[1]The development this tour describes is carried out in Lean 4 [1] with its mathematical library Mathlib [11];

For simple statements this is unsurprising. The interesting question is what happens at the other extreme. Some theorems need reasoning far stronger than ordinary arithmetic — reasoning that climbs past the natural numbers into the infinite, using principles a first course in induction never mentions. When a proof reaches that high, does the program still fall out the bottom? Or does the extra strength leave a residue — some step that asserts an object exists without ever building it — that quietly hollows the extracted program into something that type-checks but cannot actually run?

This document answers that question by opening one such extraction pipeline from end to end. We take a fixed, small formal system; we prove inside it a theorem that genuinely needs more than ordinary induction; we watch, in full detail, the proof become a program; and we then ask the proof assistant, which cannot lie, exactly which assumptions that program depends on. The tour is meant to be read start to finish by someone who has never seen any of this before. Everything is built up on the page, and each idea arrives at the moment an earlier paragraph has made you ask for it.

That is Part I, and it answers the question it sets. But it answers it about a system deliberately built to be as poor as possible: one sort of object, a short list of symbols, and everything else — ordinals, trees, sequences of moves — *coded* into a natural number. Poverty keeps the audit short, which is why we start there. It also has a price, and until you have a second system to compare against, that price is invisible. Part II builds the machine again over a language with types, runs the same theorems through it, and measures what changed. The comparison turns out to be the more interesting half: it makes proof structure, logical strength, and representation into things one can vary one at a time and watch the consequences in the extracted program.

Part I
A minimal system, end to end

Act I — The hook

Pick a natural number — say 3. Write it down. Now play the following game with it.

First, rewrite the number using only powers of 2, and rewrite the exponents that way too, and their exponents, all the way down, until nothing but 2s and small constants remain. This is called *hereditary base* 2. Now change every 2 you wrote into a 3 — bump the base — and subtract 1 from the result. Then rewrite *that* number in hereditary base 3, change every 3 into a 4, subtract 1 again. Then base 4 into base 5, subtract 1. And so on, bumping the base by one each step and subtracting a single unit each time.

The bases are marching upward without bound. Each step replaces a base by a larger one everywhere it appears — and near the start, that base change inflates the number far more violently than the lonely "subtract 1" ever pulls it back down. The sequence you are generating does not drift gently; for many starting values it erupts. Start at 4 and the first terms are those in Figure 1: the number climbs into values with no short decimal name — and yet, eventually, it comes back down to 0.

---

the extensional-collapse layer it builds on is a companion Kleene–Kreisel continuous-functionals development. Throughout, "the proof assistant" and "the machine" mean this system.

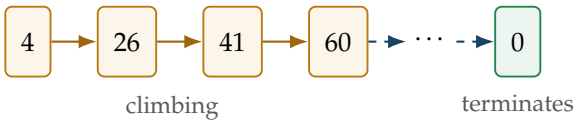


Figure 1: The Goodstein sequence starting at 4: it erupts upward, driven by the base change, then — after astronomically many steps — reaches exactly 0. (The sequence starting at 3 is tame by comparison: 3, 3, 3, 2, 1, 0.)

Goodstein's theorem [3] says that this sequence — no matter which number you started from — always, eventually, hits exactly 0.

That is already strange. A process whose every early move makes the number enormously bigger, driven downward only by subtracting one at a time, nonetheless terminates at 0 for *every* starting value. But the strange part, for our purposes, is not the theorem itself. It is what it takes to prove it, and what you can do with the proof once you have it.

Here is the first half of the surprise. You cannot prove Goodstein's theorem with ordinary mathematical induction over the natural numbers. The usual "check 0, then check that each number's truth follows from the previous one" is genuinely not enough; the theorem needs a stronger principle, one that reaches past where step-by-step induction on $\mathbb{N}$ can go. (Exactly what that stronger principle is, and why it is stronger, is Act IV's business. For now, hold onto the fact that ordinary induction falls short.)

Here is the second half. Despite needing that extra strength, the proof is *constructive* — it does not merely assert that the sequence reaches 0, it contains, buried inside it, a recipe for finding the exact step at which that happens. And in the formal system this tour is about, that recipe can be pulled out of the proof mechanically and run as an actual program. Feed it a starting number; it returns the stopping step.

Most expositions of Goodstein's theorem stop at the statement, or gesture at the proof and leave the "constructive content" as a phrase. This one opens the box all the way. We are going to watch, in complete detail and on a real example, a proof turn into a program — first on something tiny enough to hold in your hand, then on Goodstein itself, and then on a handful of other theorems that each ask the machine to do one genuinely new thing. By the end you will have seen the whole mechanism, and one question will remain: when a program falls out of a proof this powerful, does some sliver of non-constructive reasoning sneak into the program along the way? The last act is devoted to answering it.

Act II — One tiny, real example, walked through completely

Before any general theory, one concrete thing, done slowly and in full. We will not introduce a single piece of machinery in this act beyond what this one example forces us to name. No hierarchies, no levels, no collapses — those are Act III, and they will make far more sense once you have a real realizer in hand to point at.

Everything in Part I happens inside a fixed, small formal system — we will call it *the fragment*. Think of the fragment as a self-contained little world of mathematics with a rigid rulebook: a fixed vocabulary of things it can talk about (zero, successor, addition, multiplication, and a modest list of further operations we meet as we need them), a fixed grammar of statements it can form, and a fixed list of inference rules for building proofs. Nothing enters the fragment except

through that rulebook. This rigidity is deliberate: because the rules are finite and mechanical, a proof in the fragment is itself a finite, mechanical object — a tree of rule-applications — and a machine can take that tree apart. Hold that thought; it is what makes everything later possible.

Our example is the fragment's very first existential theorem — the first time it proved that *something exists* rather than that two things are equal. It is deliberately the warm-up the development itself used before tackling the general Goodstein theorem, and we use it for exactly that reason: in Act IV, Goodstein will be the grown-up version of this same statement, and you will already have watched the miniature happen by hand. The statement is: *there is a step s at which the Goodstein sequence starting from 3 has reached 0*, written in the fragment's grammar as

$$\exists s.\ \mathrm{good}(3, s) = 0,$$

where $\mathrm{good}(3, s)$ is the fragment's own name for "the value of the Goodstein sequence started at 3, after s steps."

We happen to know the answer. The Goodstein sequence starting from 3 runs $3, 3, 3, 2, 1, 0$, so it first reaches 0 at step 5. The number 5 is the *witness*: the particular s that makes $\exists s.\ \mathrm{good}(3, s) = 0$ true. Keep 5 in mind; the whole act is about watching it travel from the proof into a running program and back out.

What a realizer for this has to be

Here is the first idea we genuinely need, and it arrives because the statement forces it. The statement claims something *exists*. A proof that merely convinces us the claim is true — without ever saying *which* s works — would be useless for computing anything. So we ask: what would a proof have to carry, concretely, for us to read the answer off it?

At a minimum, it has to carry the witness — the number 5 — because without it there is nothing to return. And it has to carry, as well, some evidence that this witness really does the job: that $\mathrm{good}(3, 5)$ really is 0. This "what a proof carries" is exactly the notion of a **realizer**: the computational content of a proof, the data a machine can actually use. The shape of that data is dictated by the statement's outermost connective, and for an existential it is precisely the pair we just argued for:

> *A realizer of $\exists y.\ \varphi(y)$ is a pair x: a **witness** $\pi_1 x$, and a **certificate** $\pi_2 x$ that realizes φ at that witness. Compactly,*
>
> $$x \Vdash \exists y.\ \varphi(y) \quad \Longleftrightarrow \quad \pi_2 x \Vdash \varphi\big(\pi_1 x\big).$$

Read $\pi_1 x$ as "read the witness off x" and $\pi_2 x$ as "the rest of x." This is not a metaphor chosen for this paper; it is the actual clause the fragment uses to define what "realizes" means at an existential.

Now specialize it to our formula. Instantiating the clause at $\exists s.\ \mathrm{good}(3, s) = 0$ gives

$$x \Vdash \exists s.\ \mathrm{good}(3, s) = 0 \quad \Longleftrightarrow \quad \pi_2 x \Vdash \big(\mathrm{good}(3, \pi_1 x) = 0\big).$$

The body $\mathrm{good}(3, s) = 0$ is an *equation*, and here a second small idea appears, again forced by the example. What does it take to realize an equation? An equation is either true or not; if it is true, there is nothing further to *compute* about it — no witness to choose, no case to decide. So the realizer of a true equation carries no information at all: any object realizes it, provided the equation holds. We call such a realizer **contentless**. So the certificate $\pi_2 x$ above is contentless,

and the realizer of $\exists s.\ \mathrm{good}(3, s) = 0$ is, in effect, *just the witness* 5 wearing a trivial second component. All of the content is the witness. That is as simple as a realizer ever gets, which is why we started here.

The proof, and what extract does to it

Now we build an actual proof of $\exists s.\ \mathrm{good}(3, s) = 0$ in the fragment, in the shape the realizer analysis predicts. To prove an existential the fragment has a rule — existential introduction — that says: to conclude $\exists y.\ \varphi$, supply a specific term for y and a proof that φ holds for that term. We supply the term 5. That leaves us needing a fragment-internal proof that $\mathrm{good}(3, 5) = 0$ — and this the fragment can simply *compute*, grinding its defining equations for good on the concrete numbers 3 and 5 until it reaches 0. So the whole proof is: existential-introduction, at 5, over the computed equation, a small tree with the witness 5 at the branch point.

The fragment comes with a function — extract — that takes a finished proof and returns its realizer, by walking the proof tree and doing, at each rule, the small operation that rule's realizer calls for. On our proof it does exactly what the shape predicts (Figure 2): the witness 5 the rule supplied becomes the pair's first component; the realizer of the computed equation — contentless — becomes the second. And to run the result, we read its first component at the canonical point, recovering 5.

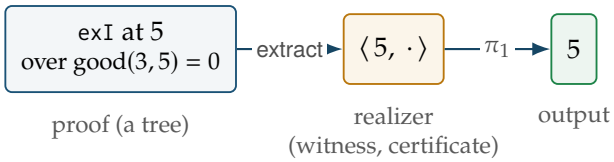


Figure 2: The whole round trip on the tiny example. The proof already contains the witness 5; extract only repackages it into the standard existential shape ⟨witness, certificate⟩, the certificate contentless because the body is an equation. Reading the first component returns 5.

That is the full round trip on the smallest possible real example: a statement was formed; we reasoned out what a proof of it must carry; we built such a proof by hand; extract repackaged its witness; evaluating the result returned 5. Nothing here required knowing what a "level" is, where realizers "live," or how any of this scales. Those questions push themselves on us the moment we ask "does this work for *every* proof?" — and Act III introduces exactly the machinery they demand.

Act III — Opening the box

We have watched one proof become one program. Two questions are now unavoidable. First: our example's body was an equation, whose realizer was contentless — so *this* realizer was essentially just a number. Real theorems have bodies with "and," "or," implications, "for all." What are realizers of *those*? Second: we ran extract once and got the right answer. Is that luck, or does extract produce a correct realizer for *every* proof? We take the questions in that order, because the second needs the first.

The fragment, formally

Before defining realizers, we should say exactly what counts as a proof in this paper. The object theory is not arbitrary Lean, not Mathlib, and not full higher-type arithmetic. It is a first-order intuitionistic theory of natural number terms, extended by a small number of named recursive functions and one transfinite-induction rule. In the Lean development this theory is the inductive family

$$\mathsf{Deriv}(\Gamma, \varphi),$$

whose inhabitants are finite proof trees from context Γ to conclusion φ. The extraction theorem quantifies over those trees.

The terms are natural-number-valued:

$$\begin{aligned} t, u \quad ::= \quad & x \mid 0 \mid S(t) \mid t + u \mid t \cdot u \mid \mathsf{pred}(t) \mid t^u \\ & \mid \mathsf{bump}(t, u) \mid \mathsf{good}(t, u) \mid \mathsf{prec}(t, u) \mid \mathsf{ord}(t, u) \\ & \mid \mathsf{hcut}(t, u) \mid \mathsf{hydra}(t, u) \mid \mathsf{hord}(t) \\ & \mid \mathsf{hcons}(t, u) \mid \mathsf{happ}(t, u) \mid \mathsf{mvcount}(t) \\ & \mid \mathsf{solves}(t, u, v, w, z) \mid \mathsf{xor}(t, u) \mid \mathsf{pas}(t, u) \\ & \mid \mathsf{look}(t, u) \mid \mathsf{fib}(t). \end{aligned}$$

Variables range only over $\mathbb{N}$. The hydras, ordinals, Hanoi move lists, Pascal entries, and finite colourings that appear later are all represented by natural-number codes. The fragment has no object-language variable of type $\mathbb{N} \to \mathbb{N}$, no lambda abstraction, and no quantifier over functions.

Formulas are generated by

$$\varphi, \psi ::= \bot \mid t = u \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \to \psi \mid \forall x.\, \varphi \mid \exists x.\, \varphi.$$

Negation is abbreviation: $\neg\varphi$ means $\varphi \to \bot$.

The logical rules are ordinary intuitionistic natural deduction: assumption and weakening; introduction and elimination for $\wedge, \vee, \to, \bot$; introduction and elimination for $\forall$ and $\exists$, with the usual freshness and capture-avoidance side conditions. The arithmetic rule is ordinary induction over natural numbers:

$$\frac{\Gamma \vdash \varphi(0) \qquad \Gamma \vdash \forall x.\big(\varphi(x) \to \varphi(Sx)\big)}{\Gamma \vdash \forall x.\varphi(x)}.$$

This is the rule used for the Fibonacci iterator, Pascal parity, Euclid, Sperner, and the ordinary recursive parts of Hanoi.

The nonlogical part is a fixed list of schema families. There are equality schemas (reflexivity, symmetry, transitivity, congruence for every function symbol), decidable equality of terms, $S(t) \neq 0$, and injectivity of S. There are defining equations for $+$, $\cdot$, predecessor, exponentiation, the Goodstein sequence, the Hydra battle, Hanoi solutions, Pascal parity, finite colour lookup, and Fibonacci. Some functions whose recursion follows the structure of coded objects enter by numeral graphs rather than by a single first-order recursion equation: hereditary base change bump, single first-order recursion equation: hereditary base change bump, ordinal comparison prec, Hydra cutting hcut, and exclusive-or xor. These graph schemas are still part of the formal theory: they are explicit constructors of Deriv, not Lean theorems silently used behind the scenes.

Finally, the fragment has one genuinely stronger induction principle: transfinite induction below ε_0, expressed over natural-number codes for ordinal notations:

$$\frac{\Gamma \vdash \forall x.\Big(\big(\forall y.\,(\mathsf{prec}(y,x) = 1 \to \varphi(y))\big) \to \varphi(x)\Big)}{\Gamma \vdash \forall x.\varphi(x)}.$$

This rule is the extra strength behind Goodstein and the Hydra termination argument. It is also the main reason the fragment is interesting: the same extraction and continuity theorem covers ordinary induction and this ε_0-recursion rule uniformly.

This puts the fragment in a slightly unusual but deliberate position. As a *language of statements*, it is weaker than Peano arithmetic or Heyting arithmetic: its variables range only over natural numbers, its function symbols are fixed in advance, and it does not contain a general theorem saying that every primitive-recursive definition may be added. As a *logic*, it is intuitionistic rather than classical, so it has no general excluded middle. As a *higher-type theory*, it is far weaker than HA^ω, because the object language cannot quantify over finite-type functionals at all. The higher types appear on the *realizer* side, not as objects quantified over by the theorem language.

At the same time, the fragment is not meant to be a weak subsystem chosen at random. It is a purpose-built extraction laboratory. It keeps the object language first-order and mechanically inspectable, but equips it with exactly the principles needed by the case studies: ordinary induction for primitive-recursive mathematics, explicit recursion equations for the coded objects under study, and one high-strength rule, transfinite induction below ε_0, for ordinal-descent arguments such as Goodstein. In that sense the comparison with PA or HA is not a single linear strength comparison. The fragment has less general arithmetic infrastructure than either theory, but it has a targeted transfinite rule that lets it prove examples ordinary PA cannot prove in its own language.

The paper's formal claim is therefore:

> *For every derivation* $D : \mathsf{Deriv}(\Gamma, \varphi)$ *in this fragment, the function* $\mathsf{extract}(D)$ *computes a modified realizer of* φ *from realizers of the assumptions in* Γ*. In particular, for every closed derivation* $D : \mathsf{Deriv}(\emptyset, \varphi)$*,* $\mathsf{extract}(D)$ *is a certified program realizing* φ*; at type level* 2 *its extensional collapse is a continuous functional.*

Everything later in the paper is an instance of this statement. Fibonacci uses ordinary induction; Hanoi uses ordinary induction plus its solution schemas; Goodstein uses the same extraction function but feeds it a proof whose recursion principle is transfinite induction below ε_0.

Realizers, for every shape of statement

In Act II we met the realizer clause for $\exists$ by asking what a proof of an existential must carry. Every connective has such a clause, arrived at the same way, and once you have seen one the rest are recognizable rather than surprising.[2] Table 1 collects them — the guiding question each time being *what would a machine need to use a proof of this?*

You have now seen the shape of every realizer the fragment will ever produce. Two of them — implication and universal — take something in and give something back; they are

[2]The interpretation is Kreisel's *modified* realizability [7]; the standard reference, whose induction-axiom realizer this development follows, is Troelstra [12].

Statement	Realizer	to use a proof of it, you. . .
$s = t$	(nothing)	. . . need nothing — it just has to hold
$A \wedge B$	a pair (r_A, r_B)	. . . get at each side
$A \vee B$	a tag + a payload	. . . first learn which side
$A \to B$	a function $r_A \mapsto r_B$	. . . feed it evidence of A
$\forall x.\, A(x)$	a function $k \mapsto r_{A(k)}$	. . . pick an x
$\exists x.\, A(x)$	a pair (witness, certificate)	. . . read the witness

Table 1: What realizes each shape of statement. An equation is *contentless*. A conjunction holds both realizers at once; a disjunction tags which side holds (0 left, non-zero right) and carries that side's realizer — structurally the same "a number, then a payload it selects" as the existential, which is why the fragment builds $\exists$ out of the disjunction's pairing machinery. The two that *consume* an input — the functions of $\to$ and $\forall$ — are the ones that do work at runtime.

the procedures, the parts that *do work* at runtime. To use a proof of "if A then B" you feed it evidence of A and it returns evidence of B; to use "for all x" you pick an x and it returns the instance. The other three — pair, tag, witness — just *hold* data. Keep that distinction in view; the next section — levels — is built entirely on it.

Levels: the cost of consuming

Here is the question the previous paragraph forced. Implications and universals are procedures — they take an input and return an output. But that input is *itself* a realizer, which might itself be a procedure taking an input, and so on. A realizer can be a procedure whose inputs are procedures whose inputs are procedures. How deep does the nesting go, and how do we track it?

The fragment tracks it with a single number on each formula, its **level** ℓ, counting how many layers of "takes an input, returns an output" a realizer can involve. The counting rule is short, and it is exactly the consume-versus-hold distinction made arithmetic:

$$\begin{aligned} \ell(s = t) &= 0, & \ell(A \to B) &= \max\big(\ell A + 1,\ \ell B\big), \\ \ell(A \wedge B) &= \max\big(\ell A, \ell B\big), & \ell(\forall x.\, A) &= \ell A + 1, \\ \ell(A \vee B) &= \max\big(\ell A, \ell B\big), & \ell(\exists x.\, A) &= \ell A. \end{aligned}$$

Only the two consuming connectives add a level. Why does $\forall$ cost a level but $\exists$ does not, when both mention a bound variable? Because $\forall y.\, \varphi$ is a *procedure* — you hand it a y and it works — while $\exists y.\, \varphi$ merely *holds* a y, the witness, already chosen. A universal consumes; an existential packages. Consuming costs a level; packaging is free. Look back at Act II: the witness 5 was already sitting in the realizer, waiting to be read; nothing had to be *run*. That is exactly why our existential example was so simple: $\exists$ adds no level. The rule is not imposed from outside — it falls straight out of the difference between a realizer that runs and one that merely stores.

Where a realizer lives: PureType

We keep saying a realizer "is a number," or "is a procedure taking a realizer and returning a realizer." Once realizers can be procedures whose inputs are procedures, we must say precisely what mathematical objects these are. Now that Act II has given us a concrete realizer, and levels have told us how deep the nesting goes, we can place them. Realizers live in a tower of types, built by that same level count:

$$\mathsf{PureType}_0 = \mathbb{N}, \qquad \mathsf{PureType}_{n+1} = \big(\mathsf{PureType}_n \to \mathbb{N}\big).$$

The bottom floor is the natural numbers — where a bare witness like our 5 lives. Each floor up is the functions from the floor below to the numbers (Figure 3). Each time a formula's level rises by one — each time you add an implication or universal that *consumes* a realizer — the realizer climbs one storey: a realizer of a level-n formula lives n storeys up. This is why we spent so long on levels first: the level is the floor number.

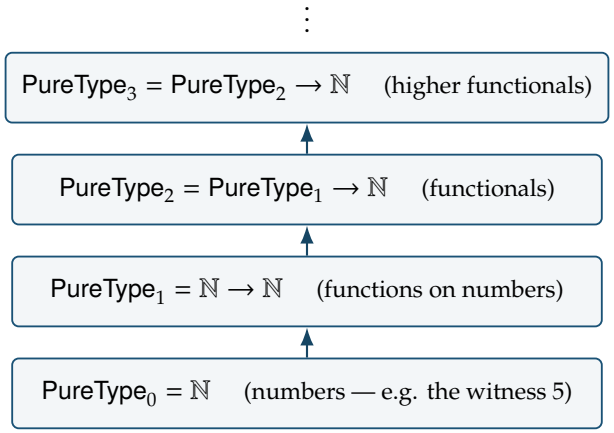


Figure 3: The PureType tower. A realizer of a level-n formula lives on floor n. Our Act II realizer was a level-0 object — a number — because its formula, an existential over an equation, adds no level. This tower is built by completely elementary means, with nothing classical or non-constructive anywhere in it — a fact Act V will turn on.

The program a closed statement denotes: CtQ

Two *different* proofs of the same statement can produce two different realizers — different objects in the tower — that nonetheless compute the same answers everywhere. To speak of "*the* program" a theorem denotes, we collapse realizers that agree as functions into single objects. For a closed statement, its realizer lands, after this collapse, in a space the development calls CtQ, and its image there is the one *proof-independent* program the theorem denotes:

$$\boxed{\text{Realizer}} \xrightarrow{\text{collapse}} \boxed{\mathsf{CtQ}}.$$

We name CtQ now but do not yet lean on it: for the collapse to be the honest space of functionals rather than an arbitrary quotient, the realizers landing in it must be *continuous*, a property we state below. Read CtQ for now as a promissory note — *the program of a closed theorem is a single*

proof-independent object, once we know its realizer is continuous — redeemed two definitions on.[3]

Soundness: it works for every proof

Now the second opening question. We ran extract once and got a correct realizer. Is that always so? It is — and it is the central theorem of the whole construction:

> ***Soundness.*** *For every proof the fragment can build, of every statement,* extract *returns a genuine realizer of that statement.*

What we watched happen once, by hand, for $\exists s.\ \mathrm{good}(3, s) = 0$, soundness guarantees for *every* derivation the fragment admits. Its proof is itself an induction — not over numbers, but over the shape of proofs. There are finitely many inference rules; soundness checks, once, that each rule's piece of extract does the right thing *assuming the pieces for its sub-proofs did*. Because every proof is a finite tree of those rules, checking each rule once covers every proof there will ever be. From here on we may speak of "the program a theorem extracts to" and know, without further checking, that it is correct at every input.

Generic continuity: every program is continuous

One more property, stated now and explained more deeply once Act IV gives us programs worth explaining it for. A realizer high in the tower is a functional — it takes functions as inputs. The pathological ones read *infinitely much* of their input before deciding an output, and those are exactly the objects that do not collapse honestly into CtQ. The good functionals are the **continuous** ones: to determine any single output they consult only *finitely much* of their input. The development proves, once and generically:

> ***Generic continuity.*** *Every realizer* extract *produces, from every proof in the fragment, is continuous.*

This redeems the promissory note on CtQ: because every extract is continuous, every closed theorem's realizer collapses honestly into a single CtQ element — one honest program. We will see *why* generic continuity holds in Act VI; the harder examples of Act IV should exist first to make that "why" worth the space. The box is open: we have realizers for every shape, the level accounting, the tower they live in, the collapse into programs, and the two guarantees — correctness and continuity. Everything from here runs on exactly this machinery, unchanged. What varies is only the proofs we feed it. In the implementation, those proofs are explicit `Deriv` trees of the fragment; ordinary Lean theorems become input to this pipeline only after they have been represented and proved in that object language.

Act IV — Running the same machine on harder proofs

Nothing new gets added to the extractor in this act. The pipeline — prove a theorem, run extract, get a certified program — is fixed. What changes is the difficulty of the proofs, and each example is chosen because it forces exactly one genuinely new capability out of the same fixed

[3] CtQ is the extensional collapse of the Kleene–Kreisel continuous functionals [6, 7]; for a modern account see Longley and Normann [8].

machine. We name that capability in a heading, so the staircase is visible (Figure 4). Before the first step, one rule the first five examples all lean on.

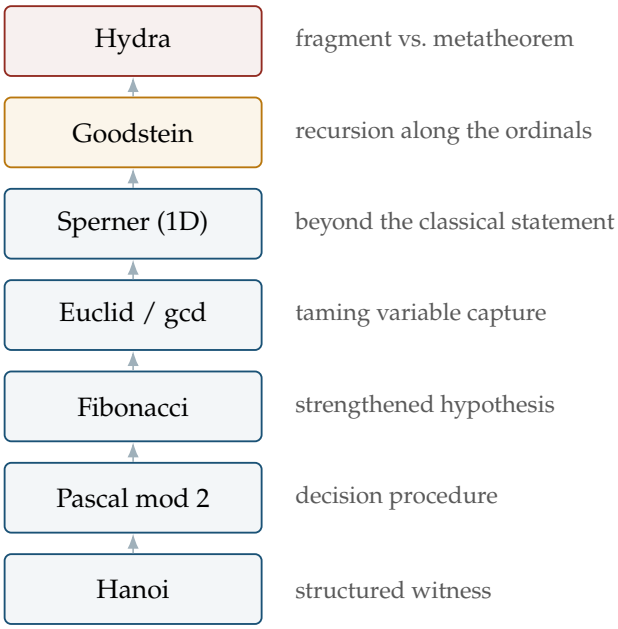


Figure 4: The staircase of Act IV: each example is the one below plus a single new demand on the same fixed extractor. Amber marks the step up to transfinite recursion (Goodstein); red marks the boundary of what the fragment can even express (Hydra).

The rule that becomes recursion: ordinary induction

The fragment has ordinary mathematical induction as an inference rule — call it `ind`. It says the familiar thing: to prove $\forall x.\, \varphi(x)$, prove $\varphi(0)$ and prove that $\varphi(x)$ implies $\varphi(x+1)$. Watch what extract does to a proof that uses it. The base $\varphi(0)$ extracts to a realizer — the answer at 0. The step extracts to a *transformation* — a procedure turning a realizer of $\varphi(x)$ into one of $\varphi(x+1)$. To get the realizer at a number k, extract starts from the base and applies the step k times:

$$R(0) = r_0, \qquad R(n+1) = f\big(R(n)\big).$$

That is precisely a recursive program: a base value and a function iterated as many times as the input says. A proof by induction, run through extract, *is* a program by recursion (Figure 5) — the shape underneath all five of the next examples.

Figure 5: Extraction turns the base and step of an induction into the base and step of a recursion. The stronger induction of Goodstein (below) turns, by the same move, into a stronger recursion.

New capability: a structured witness (Towers of Hanoi)

The Towers of Hanoi puzzle asks for a sequence of moves transferring n disks between pegs. The fragment proves such a sequence always exists — and, in the same breath, that it has exactly $2^n - 1$ moves. What is new is the *shape* of the witness. In Act II the witness was a bare number, 5. Here the thing whose existence we prove is a whole *sequence of moves* — structured data, encoded as a number but meaning a list. When extract runs, the realizer's witness is that move sequence, and it decodes into a readable list of "move a disk from this peg to that peg" instructions — the classical optimal solution, produced by the extracted program and checked to be exactly the moves a human would write. The existential machine of Act II, unchanged, now returns structured data. (The proof uses `ind` on the disk count, so it extracts to a recursion; the recursion branches — two smaller sub-towers joined by one move — which is why the count comes out $2^n - 1$.)

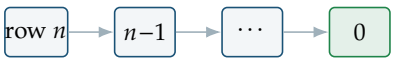
Every recursion in this act halts for the same underlying reason, and it is worth naming that reason once, here, where it is easiest to see. The disk count is a *descent certificate*: a quantity that strictly drops at every recursive call and, being a natural number, cannot drop forever (Figure 6). Each example below carries one; watch what it is.

descent certificate: disks remaining

Figure 6: Why recursion terminates (Hanoi). Each recursive call solves a tower of strictly fewer disks, so the disk count — the *descent certificate* — falls to 0 and can fall no further. The recursion *branches* (two sub-towers per disk), but every branch drops the count, so the whole tree of calls is finite.

New capability: a decision procedure (Pascal's triangle, mod 2)

Replace every entry of Pascal's triangle by its remainder mod 2: every number becomes a 0 or a 1. The fragment proves, for each position, that its parity is 0 or 1 — and, more to the point, *which*. The new capability is that the extracted program is a **classifier**. The statement is a disjunction, "this entry is 0, or it is 1," so (Table 1) its realizer carries a tag naming the answer. Run extract and that tag becomes a runnable function from a position to its parity — a decision procedure, produced not by us writing one but by extracting it from a proof that merely said "the answer is one or the other." Tabulate that classifier's outputs — a mark for 1, a blank for 0, row by row — and the picture that appears is the Sierpiński triangle (Figure 8). There is no gap between the picture and the theorem: every cell of the drawing is one instance of the proved disjunction, its value filled in by the extracted realizer's own tag.



descent certificate: the row index (outer), the column index (inner)

Figure 7: Why recursion terminates (Pascal). The proof is a *nested* induction: the outer recursion descends the row index, and within each row an inner recursion descends the column index. Two descent certificates, both ordinary natural numbers, one inside the other.

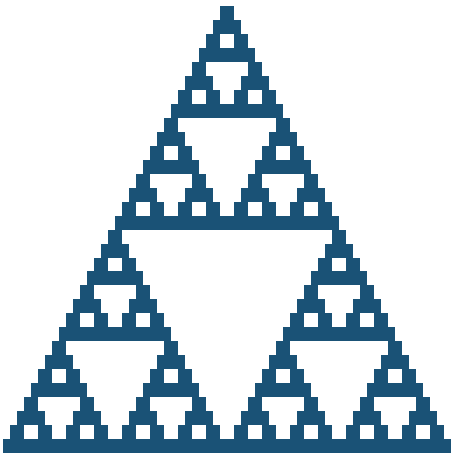


Figure 8: The extracted classifier's own output, rows 0–31: a filled cell where the decided parity is 1, blank where it is 0. The Sierpiński triangle here is not a picture drawn to resemble the theorem — it *is* the theorem, one cell per proved instance of "this entry is 0 or 1," each cell's value read off the extracted realizer's tag. Generated from the parity recurrence $\mathrm{pas}(n,k) = \mathrm{pas}(n{-}1,k{-}1) \oplus \mathrm{pas}(n{-}1,k)$ and cross-checked against $\binom{n}{k} \bmod 2$; the $243 = 3^5$ filled cells over 32 rows are exactly the count the closed form predicts.

New capability: a strengthened induction hypothesis (Fibonacci)

The Fibonacci numbers $0, 1, 1, 2, 3, 5, 8, \ldots$ are each the sum of the previous two. Suppose we want to prove, in a form from which we can extract a program, that fib is total. Try it by ordinary induction and you hit a wall worth feeling: induction hands you the statement *at* n, but computing $\mathrm{fib}(n+2)$ needs *two* earlier values, $\mathrm{fib}(n+1)$ and $\mathrm{fib}(n)$, and the hypothesis at n alone does not give both. The new capability is the standard cure. Instead of proving "$\mathrm{fib}(n)$ exists," prove the stronger pair statement: the *pair* $\big(\mathrm{fib}(n), \mathrm{fib}(n+1)\big)$ exists. Now the hypothesis hands you both consecutive values, and the step is a single shift,

$$\big(\mathrm{fib}(n),\ \mathrm{fib}(n+1)\big) \longmapsto \big(\mathrm{fib}(n+1),\ \mathrm{fib}(n) + \mathrm{fib}(n+1)\big).$$

A stronger hypothesis is a stronger thing to be handed at the step, so the induction goes through. Extract that proof and — exactly as Figure 5 predicts — you get the fast two-variable iterative Fibonacci every programmer knows: carry a pair, shift it n times, read off a component. The proof's strengthened hypothesis *is* the program's pair of accumulator variables; you can watch the extracted pair advance through $(0,1), (1,1), (1,2), (2,3)$ — the loop's running state, read out of a proof that never mentions a loop.

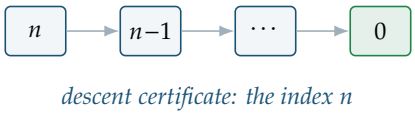


Figure 9: **Why recursion terminates (Fibonacci).** Strengthening the hypothesis to a pair changed what is *carried* at each step — the two accumulators — not what makes the recursion stop. The descent certificate is still the plain index n, counting down to 0.

New capability: taming variable capture (Euclid's gcd)

The fragment proves that any two numbers have a greatest common divisor — and the extracted witness *is* the gcd, computed by subtractive Euclid (repeatedly replace the larger number by the difference), with no gcd operation built into the fragment. Two supporting economies make this possible. First, the fragment never added an "order" operation: $s < t$ is simply $\exists d.\,(s+1)+d = t$ — order is an existential over addition, not a primitive. Second, it never added "strong induction" (where the step may use *all* smaller cases); that is *derived* from ordinary `ind`.

But the new capability the gcd proof forces is neither of these — it is a problem about *substitution*. When you instantiate an induction hypothesis or a "for all," you substitute a term for a bound variable; if that term mentions a variable that is *also* bound in the formula, the substitution silently confuses two different variables sharing a name — the notorious *variable capture*. The fragment's substitution is deliberately simple-minded: rather than quietly renaming to avoid capture, it *forbids* the capturing substitution outright, so the proof author must route around it. Euclid's recursion is the first place two capture problems bite at once, and the first to use the fragment's two general-purpose dodges *together*: give the offending term a fresh name with an extra "for all" and instantiate at the harmless name; and, when a recursion permutes its arguments, rename those variables to fresh ones first. Neither changes what is proved or what the program computes; both merely coax a naive substitution system into accepting a substitution that is morally fine. Out the far end comes a certified greatest-common-divisor calculator — which, pleasingly, needs no special case for zero: the specification is met by $\gcd(0,0) = 0$, since everything divides 0.

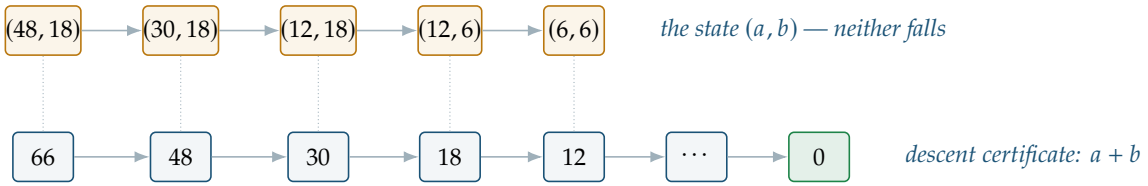


Figure 10: Why recursion terminates (Euclid). Subtractive Euclid replaces the larger number by the difference. Neither coordinate of the pair (amber) falls at every step — but their *sum* (below) does, strictly, until it reaches 0. The descent certificate here is not one of the arguments; it is the derived quantity $a + b$, which is exactly why the proof runs a course-of-values ("strong") induction *on the sum* rather than on either number.

New capability: more general than the classical statement (Sperner, 1D)

Colour the points $0, 1, \ldots, n$ along a line, each some natural-number colour, with the two ends coloured differently. Then somewhere two *adjacent* points must differ — you cannot get from one end-colour to the other without a change. This is the one-dimensional Sperner lemma [9], the discrete ancestor of the intermediate value theorem. The fragment proves it by ordinary induction, scanning the line and carrying "either we are still at the start colour, or we have already found a crossing"; the extracted program is a left-to-right search returning the *first* crossing. But the new thing is not the extraction — it is the theorem. The classical statement is usually given for *two* colours; the fragment's proof never uses that restriction. It works for *arbitrary* natural-number colourings, so the extracted search is more general than the classical picture. The proof turned out to prove *more* than the theorem it was written for.

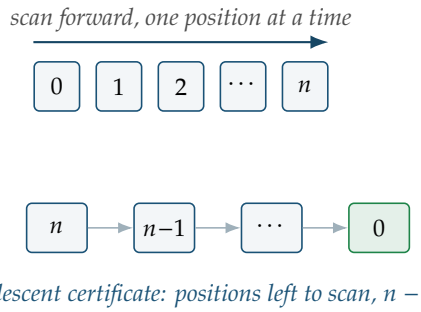


Figure 11: Why recursion terminates (Sperner). The extracted program is a *forward linear scan*: it walks the points $0, 1, \ldots, n$ left to right, stopping at the first colour change. The descent certificate is the number of unscanned positions, $n - m$, dropping by exactly one per step — a straight walk to the end, not an interval halved by bisection.

New capability: recursion along the ordinals (Goodstein)

Now Goodstein itself, the general theorem behind Act II's single hand-worked case. Act II proved $\exists s.\ \mathrm{good}(3, s) = 0$ with the witness supplied by us as 5. Goodstein's *theorem* is the universal version,

$$\forall m.\ \exists s.\ \mathrm{good}(m, s) = 0,$$

and we want from it the same thing Act II gave, but uniform in m: a program that, given m, returns the stopping step. The proof cannot know m's answer in advance, so the witness must be *produced* by the proof.

Every example so far has handed us a descent certificate that was an ordinary natural number — a disk count, a row index, a plain index, a sum, a count of positions left to scan. **Goodstein is where that stops being enough.** This is the same fact as Act I's promissory note — that ordinary induction is not enough — now come due: induct on m, or s, or the value $\mathrm{good}(m, s)$, and you fail, because the value goes *up* at almost every step, so no natural-number quantity decreases as the sequence runs. The descent certificate Goodstein needs is not a natural number at all — it is the same idea of descent, reaching one step past the naturals. To see why, write a number in hereditary base and watch a step act on it:

$$13 \;=\; 2^{2+1} + 2^2 + 1 \;\xrightarrow{2 \rightsquigarrow 3}\; 3^{3+1} + 3^3 + 1 \;=\; 109 \;\xrightarrow{-1}\; 108.$$

Replacing every base 2 by a 3 — *including the ones inside the exponents* — inflates 13 to 108 in a single step. Yet if you read that same expression not as a number but as an *ordinal* — interpret each base as ω, the first infinity — the base change does nothing (every base is already ω), while the "-1" strictly *lowers* it. Every Goodstein step, however it inflates the natural-number value, strictly descends an ordinal below ε_0 (epsilon-nought); and the ordinals are *well-ordered* — no infinite descending chains (Figure 12) — so the sequence cannot descend forever and must reach 0.

$$0 < 1 < 2 < \cdots < \omega < \omega + 1 < \cdots < \omega^2 < \cdots < \omega^\omega < \cdots < \varepsilon_0.$$

This is the extra strength Act I warned of: **transfinite induction up to** ε_0, the single rule in the fragment's whole repertoire that is not a consequence of ordinary logic and ordinary induction — it is exactly the principle Gentzen showed lies beyond ordinary arithmetic [2], here realized as

recursion along a primitive-recursive ordering in the manner of Tait [10]. To make it mechanical the fragment builds the ε_0 ordinals *as ordinary numbers* under a hand-crafted encoding, defines the "smaller ordinal" relation on those codes, and proves — by elementary means, no infinite reasoning — that it admits no infinite descending chain. (Why the encoding was built by hand rather than borrowed is an Act VI discovery.)

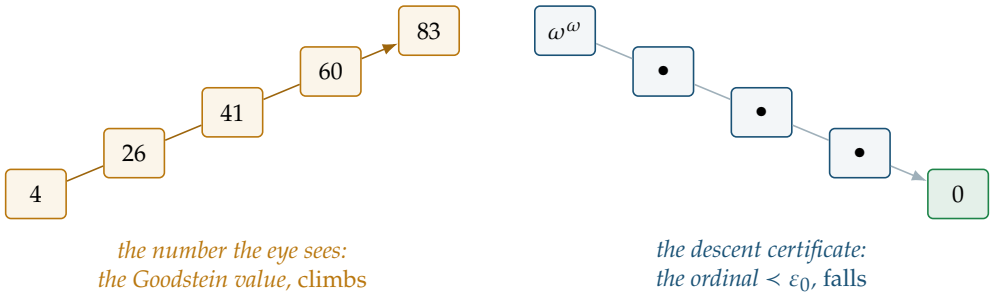


Figure 12: Why recursion terminates (Goodstein) — the signature contrast. Starting from $m = 4$, the Goodstein value climbs (left; the real values $4, 26, 41, 60, 83, \ldots$) while the ordinal assigned to each state (right) strictly *descends* through the well-order below ε_0, from ω^ω toward 0. The intermediate ordinals (shown •) are intricate, but every one lies strictly below the last, and a descending chain of ordinals cannot run forever. The proof follows the falling certificate on the right, never the climbing number on the left — which is why a sequence that grows can still be proved to stop.

Now the two acts line up. In Act II, extract turned an existential proof into a witness by reading a number the proof supplied. Here it turns Goodstein's proof into a witness by *recursion along the ordinals*: the transfinite-induction rule extracts to a program that descends the ordinal of each state until it reaches the bottom, and the number of descents is the stopping step. Where `ind` extracted to "iterate a step k times" (Figure 5), the transfinite rule extracts to "recur along the ordinal ordering" — a stronger recursion for a stronger induction. Same machine, one gear higher:

$$\forall m.\, \exists s.\ \mathrm{good}(m, s) = 0 \quad \xrightarrow{\text{extract}} \quad m \longmapsto s.$$

One theorem; one extracted program. And it runs (Table 2). The row for $m = 4$ is where Act I's "numbers explode" stops being an assertion: the program declines to finish because the terminating sequence really is that long.

start m	0	1	2	3	$\cdots$	4
stopping step s	0	1	3	5	$\cdots$	(not attempted)

Table 2: The extracted stopping-step program $m \mapsto s$. It returns 0 and 1 directly when run; soundness certifies 3 and 5 for $m = 2, 3$ (the sequences $2, 2, 1, 0$ and $3, 3, 3, 2, 1, 0$) even where running the program that far is slow; $m = 4$ is not attempted — its sequence begins $4, 26, 41, 60, \ldots$ and its stopping step is astronomically large.

An aside: the same function, computed two ways

That stopping-time function $m \mapsto s$ can be written as a program in two very different ways, and the difference is exactly the subject of this whole tour. The obvious way is a *search*: run the

sequence, test for zero, return the first step that hits it — a `while` loop, the classical minimization operator. It is easy to write and, for small m, easy to run. But look at where its termination lives: the loop halts *if and only if* the sequence really does reach 0 — which is Goodstein's theorem. Nothing inside the loop guarantees it. The other way is the *extracted* program of Figure 13: the recursion along the descending ordinals we have just described, which halts *by construction*, because a chain of ordinals below ε_0 cannot descend forever — no matter how large the natural-number values swell in between.

A — search (the `while` loop)

$s \leftarrow 0$
`while` good$(m, s) \neq 0$: $s \leftarrow s + 1$
`return` s

halts *iff* the sequence reaches 0
that is Goodstein's theorem
in Lean: a `partial def` — *not* certified

B — ordinal descent (extracted)

recurse along $\prec$ from the ordinal of good$(m, \cdot)$; each step lowers it below ε_0;
`return` the number of steps

halts *by construction* (well-founded)
termination is intrinsic
in Lean: a total `def` — certified

both return the same column: $0 \mapsto 0$, $1 \mapsto 1$, $2 \mapsto 3$, $3 \mapsto 5$, ...

Figure 13: One function, two programs. Both compute the Goodstein stopping time and agree wherever they run, but their termination guarantees sit in opposite places. Program A is a search whose halting *is* the theorem: written directly in Lean it must be marked `partial`, the kernel's way of saying "not certified to terminate." Program B, extracted from the proof, is a total `def` whose halting is carried by the well-founded ordinal descent. The gap is one keyword. Appendix A shows Program B as a tree.

The dependency is not merely narrated — it can be made a proof. Lean's constructive minimization operator (`Nat.find`) turns the search into a *total* function, but only when handed a proof that a zero exists; and the one such proof available is the extracted theorem itself.

> *The search becomes certified-total exactly by* consuming Goodstein's theorem *as its termination argument. "Halts iff the theorem holds" stops being a remark and becomes the type of the program.*

One caveat keeps the picture honest: the certificate buys *termination*, not *speed*. The certified search still evaluates by running the sequence, so at $m = 4$ it is as unreachable as before — Program B's guarantee is that it *will* finish, not that it finishes soon.

New capability: a fragment theorem versus a metatheorem (the Hydra)

The last and hardest example marks a boundary — the place where what the fragment can *prove* and what it can even *state* come apart.

The Hydra is a many-headed tree; Hercules chops a head and it regrows, often growing *more* heads than it lost. The Kirby–Paris theorem [5] says Hercules wins anyway: every battle terminates. The reason is Goodstein's in disguise — each chop, though it grows the tree, strictly lowers an ordinal below ε_0 assigned to it. The fragment can prove termination for *one fixed, encoded battle*: fix a starting Hydra and a fixed way of playing, code it all as numbers, and prove $\forall h.\ \exists t.\ \text{hydra}(h, t) = 0$ — the exact shape of Goodstein's theorem, proved the exact same way, extracting to a battle-length program. Business as usual — including the reason it terminates (Figure 14).

But the *real* Kirby–Paris theorem says Hercules wins *no matter what strategy* he uses — and

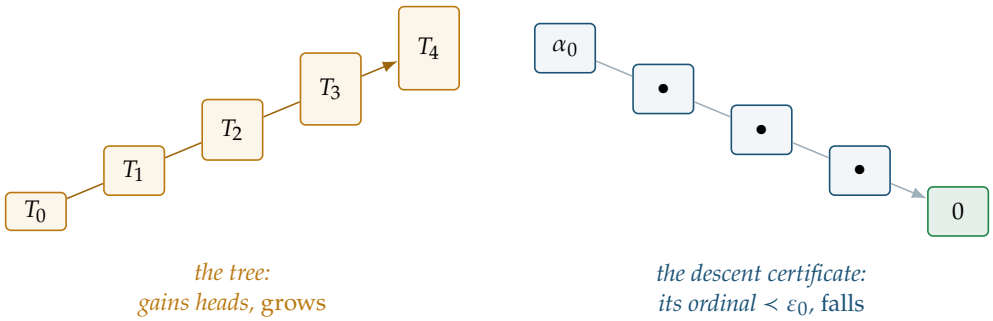


Figure 14: Why recursion terminates (the Hydra) — the same signature. Each chop regrows the Hydra, usually with *more* heads than before, so the tree size climbs (left, successive states $T_0, T_1, \ldots$). Yet the ordinal $\alpha_i \prec \varepsilon_0$ assigned to each tree strictly falls (right), by exactly Goodstein's mechanism. Termination follows the ordinal, not the head count — the identical shape to Figure 12, which is why the fragment proves it the identical way.

this the fragment cannot even express (Figure 15). To say "for every strategy" you must quantify over strategies, and a strategy is a *function* (it looks at the current Hydra and decides the next chop). The fragment's "for all" ranges only over *numbers*, never functions. The limitation is not that the fragment fails to *prove* the general theorem; the sentence expressing it is not in the fragment's language at all. So the general theorem is proved *outside* the fragment, in the ordinary mathematics of the proof assistant, which quantifies over functions freely; and the fragment's single battle is checked to be one of those general plays, a genuine special case. This split — a *fragment theorem* for one battle, a *metatheorem* for all strategies — is forced, cleanly and provably, by the one fact that the fragment has no function variables. It is the right place to stop feeding the machine harder proofs.

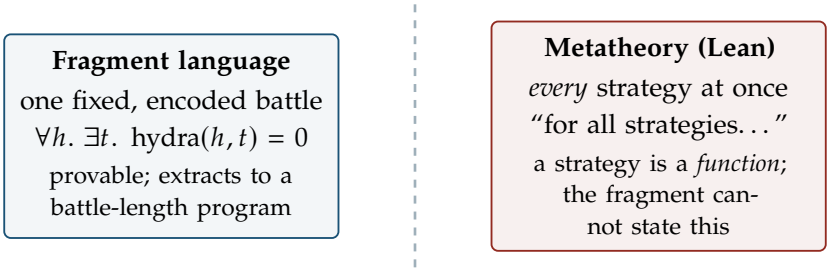


Figure 15: The boundary the Hydra draws. Left of the dashed line, the fragment proves — and extracts a program for — termination of one encoded battle. Right of it, the fully general claim, which the fragment cannot even write down, so it is proved directly in the metatheory. The line is forced by the absence of function variables, not chosen.

Read the last column top to bottom: five ordinary natural numbers, then two ordinals. That is the whole of what changes across the act. Ordinary induction and transfinite induction are not two kinds of computation — they are the one extraction pipeline reading two *notions of descent*, the natural numbers and the ordinals below ε_0; and in every row of the table, a proof that some certificate descends is what extract turns into a recursive program that halts.

Case	Theorem (in the fragment)	Extracted program	Descent certificate
Hanoi	$\exists k.$ Solves $\wedge$ len=2^n-1	move sequence	disk count n
Pascal	$\forall n, k.\ \mathrm{pas}(n,k) \in \{0,1\}$	parity classifier	row index, then column
Fibonacci	$\forall n.\ \exists y.\ \mathrm{fib}\, n{=}y$	pair iterator	index n
Euclid	$\forall a, b.\ \exists g.\ g \mid a,\ g \mid b, \ldots$	gcd calculator	sum $a + b$
Sperner	$\forall n, c.\ \exists k{<}n.\ c_k{\neq}c_{k+1}$	first-crossing search	positions $n - m$
Goodstein	$\forall m.\ \exists s.\ \mathrm{good}(m,s){=}0$	stopping-step function	ordinal $\prec \varepsilon_0$
Hydra	$\forall h.\ \exists t.\ \mathrm{hydra}(h,t){=}0$	battle-length function	ordinal $\prec \varepsilon_0$

Table 3: Act IV in one table. One row per case study: the theorem proved inside the fragment, the program extract produces from it, and the descent certificate that makes that program terminate. The middle rule separates the five whose certificate is an ordinary natural number from the two — Goodstein and the Hydra — whose certificate is an ordinal below ε_0.

Act V — What it costs: the constructivity audit

We have now watched proofs of real strength become running programs. Goodstein and the Hydra needed transfinite induction up to ε_0 — genuinely more than ordinary arithmetic. And the programs are not toys: an ordinal-descending recursion, a first-crossing search, a greatest-common-divisor calculator, a parity classifier.

So here is the suspicion any careful reader should now be holding, and it is the right one. Proofs of this strength, in ordinary practice, routinely use *non-constructive* reasoning — the law of excluded middle, the axiom of choice, arguments that assert an object exists without building it. Surely, somewhere in a construction reaching all the way to ε_0, some non-constructive step sneaks in. And if it does, the extracted "program" might be a fiction — an object that type-checks but cannot run, its content secretly resting on an axiom that computes nothing. The whole promise of the tour would collapse at its most impressive point.

This act resolves the suspicion, precisely, in the program's favour — and the resolution is possible only because the proof assistant tracks, mechanically and unforgeably, exactly which axioms any object depends on. The suspicion has a specific name. The genuinely non-constructive axiom — the one that asserts choices can be made with no rule for making them, and from which the full law of excluded middle follows — is called `Classical.choice`. It is the axiom whose presence in a program would hollow it out, turning a thing that computes into a thing that merely type-checks. So the question has one precise form: does `Classical.choice` appear in the list?

Now the finding, stated exactly as the machine reports it. Every extracted program in the fragment — the Goodstein stopping-step function, the Hydra battle-length function, the gcd calculator, the Sperner search, the Pascal classifier, the Fibonacci iterator, and the extraction machinery itself — reports its axiom dependency as exactly

```
[propext, Quot.sound]
```

and `Classical.choice` is *not* on the list. Not "we believe"; not "morally." The kernel, asked directly, lists those two names and not the third, for every object that actually runs.

So what *are* the two names that are there? Neither is the one the suspicion feared. `propext` and `Quot.sound` are the proof assistant's own two foundational axioms, and both are constructively harmless: neither lets you conjure an object you cannot build, neither implies excluded middle. They are bookkeeping about when two propositions, or two quotient elements, count as equal — part of the machine's floor, computationally inert. The trusted base of every extracted computation is the kernel together with these two, and nothing classical.

Does `Classical.choice` appear *anywhere*? Yes — and where it does is a precise boundary (Table 4, Figure 16). It enters in exactly two places, and neither runs. It appears in some *correctness proofs* — the proofs that the programs meet their specifications — and even there only through two humble library lemmas about updating a function at a point; these live entirely among propositions and are used to *prove a program correct*, never inside it. And it appears in the final *packaging* step — the collapse of a realizer into its single CtQ element — performed once to say "here is the one program this theorem denotes," inheriting choice from the general theory of extensional functionals. But again: the packaging is *about* the program; the program itself is the choice-free realizer that got packaged.

Component	**Uses `Classical.choice`?**
Extracted programs (Goodstein, gcd, . . .)	**no**
The extraction pipeline itself	**no**
Some correctness proofs	limited — two library lemmas
The CtQ packaging step	yes

Table 4: Where the one non-constructive axiom does and does not appear. Everything that runs is choice-free; `Classical.choice` is confined to things that only reason *about* the programs.

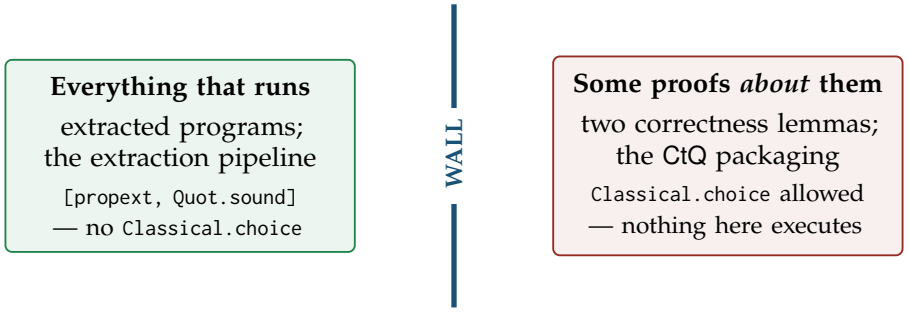


Figure 16: The constructivity wall. On one side, everything that executes — choice-free, resting only on the inert kernel axioms. On the other, some proofs *about* those programs and one packaging step, which may use choice freely because nothing there ever runs. The one non-constructive axiom in the whole development is quarantined on the far side from anything that computes.

A proof needing strictly more than Peano arithmetic — transfinite induction to ε_0 — became a program, and the program is *constructive to the last axiom the machine can name*. The strength went into proving the theorem; none of it leaked into the thing that runs.

Act VI — What the machine forced us to discover

A proof assistant is usually cast as a checker: you have the argument, it confirms you made no mistake. That is the smaller half of what happened here. The larger half is that the machine, by refusing to accept certain things, *forced* decisions that would never have surfaced on paper — where a wave of the hand carries you past exactly the spots where it dug in. Four episodes; each is a place where the development is shaped the way it is because the machine would not let it be otherwise.

The level bookkeeping was not designed — it was extorted

Act III's realizers live in a tower of types, and each formula carries a level saying how high in the tower its realizers sit. The rule turned out to be sharp:

> *Connectives that* consume *a realizer — implication and "for all" — cost a level. Connectives that merely* package *realizers — "and," "or," and "there exists" — cost nothing.*

On paper one might guess a level discipline and tune it until the proofs go through. Here there was no room to guess: assign "there exists" a level it should not have, or deny "for all" the level it needs, and the realizer types fail to line up and the type-checker rejects the extractor outright — not with a warning, with a hard error at the exact malformed clause. The rule above was read off the pattern of what the machine accepted and what it refused. The distinction it encodes — consuming versus packaging — is a real conceptual fault line in realizability, and it was the type-checker that drew it.

The Hydra boundary was provable, not merely felt

Act IV ended at the wall where the general Kirby–Paris theorem cannot be stated in the fragment. On paper that is easy to slide past — one writes "for every strategy…" and reads it as innocent. Inside the machine it is not innocent: to write it you must produce a term quantifying over functions, and the fragment's grammar has no such term former. The attempt does not produce a weak or unprovable statement; it produces *no statement at all*, because it does not parse. The split between the fragment theorem (one battle) and the metatheorem (all strategies) is therefore not a matter of taste about where to stop — it is forced by a checkable property of the fragment's syntax. The machine turned "this feels out of reach" into "this is provably inexpressible."

The pairing had to be hand-built, or the audit would have failed

Act V's clean verdict — nothing that runs uses `Classical.choice` — was nearly lost to a detail three layers down. The ε_0 ordinals are encoded as ordinary numbers, and that encoding needs a way to pack two numbers into one: a pairing function. The proof-assistant library ships one, ready to use. But its supporting lemmas are, throughout, proved with `Classical.choice` — and because the transfinite recursor's very *definition* calls the pairing, that choice would have flowed downstream into extract and tainted *every* extracted program (Figure 17). The paper argument "we encode ordinals as numbers" hides this completely; `#print axioms`, run on an early prototype, made it glaring — `Classical.choice` appearing in a program that should have had none, traced back to the borrowed pairing. The fix was to hand-roll a triangular pairing

that the kernel can compute and whose lemmas are choice-free. A foundational decision, invisible on paper, forced into the open by the audit the machine made possible.

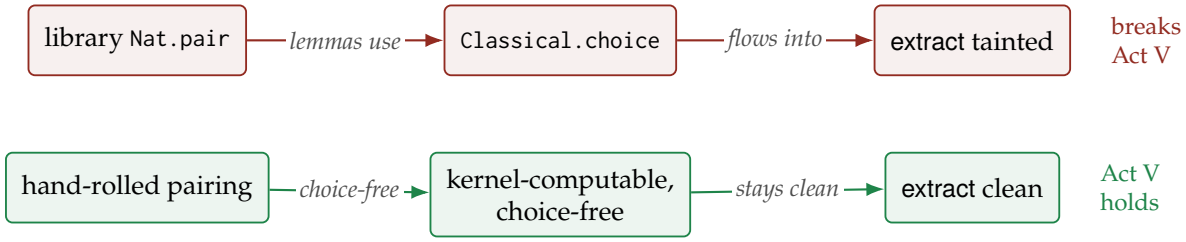


Figure 17: The discovery the audit forced. Reusing the library's pairing (top) would have threaded `Classical.choice` through the ordinal recursor into every extracted program. Hand-rolling a choice-free pairing (bottom) keeps the one non-constructive axiom out of everything that runs. The leak was invisible on paper and glaring under `#print axioms`.

A verified optimization that changed nothing — and we kept it

The transfinite recursor is the development's most expensive machine, so an obvious improvement suggests itself: *memoize* it — cache each ordinal's result so it is not recomputed. This was implemented, proved correct, and wired in as a drop-in replacement for the plain recursor. Then it was measured, and it changed *nothing*. The reason is precise and only visible once you look: producing the recursor's value at an ordinal code is already a constant-time operation — it hands back a closure — so the real cost lives in the *applications* of that closure, which a table keyed by the ordinal code can never reach. The verified-but-useless memo path is kept in the development, with its correctness proof, as the record of a negative result that was *measured* rather than assumed. On paper an optimization that looks plausible tends to get written up as a win; here the profiler said otherwise, and honesty about the null result was cheaper than the illusion of a speedup.

The descent was demoted from assumption to theorem

The engine of Goodstein and the Hydra is one fact: each step strictly lowers the ordinal. In an early version that fact was simply *assumed* — imported as an axiom schema and used. That is a real dependency, and the machine records it: an assumed descent appears on the `#print axioms` list, in plain sight, for anyone to see how much is being taken on faith. The development later *earned* it back, deriving the descent from three small schemas, each about a single symbol in relation to a single ordinal operation, none mentioning Goodstein or the Hydra at all. The gain is not that the proof got shorter — it is that the ledger of assumptions got smaller, and the machine is what keeps that ledger honest. What you assume is visible; reducing it is progress you can point to, not a matter of narrative.

Act VII — What the small system shows

Set the examples back-to-back — Hanoi, Pascal, Fibonacci, Euclid, Sperner, Goodstein, the Hydra — and the natural summary is "look, one pipeline turns proofs into programs." But that pipeline is old news: that a constructive proof carries computational content is the content

of realizability and of the Curry–Howard correspondence [4], both decades established. The reader was told as much in Act I, and being shown a known correspondence at work is not, by itself, a surprise — though Figure 18 makes it unusually concrete, reading the extracted program back onto the proof of the hardest theorem here.

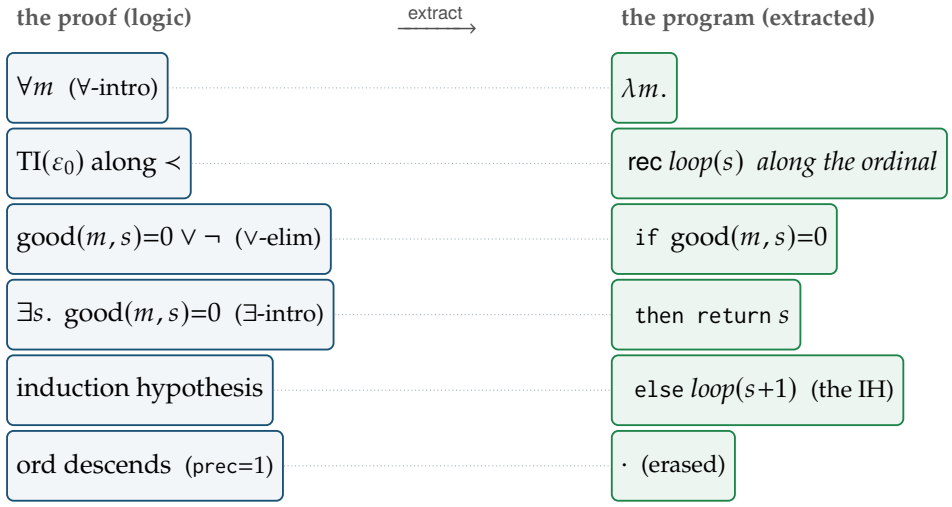


Figure 18: The extraction map on the hardest theorem. Modified realizability is Curry–Howard with the map made explicit: extract sends each proof rule (left) to one program operation (right). A witness supplied by hand extracts to a constant — $\exists s.\ \mathrm{good}(3, s){=}0$, proved by `exI` 5, gives `return` $\langle 5, \cdot\rangle$ — while a witness produced by transfinite induction extracts to a recursion, the shape of the induction becoming the shape of the recursion. The induction hypothesis *is* the recursive call; the ordinal descent, carrying no runtime content, is erased. The same object is a proof that every Goodstein sequence terminates and a program that computes when. What is old is the correspondence; what is new is that this one fixed apparatus reaches $\mathrm{TI}(\varepsilon_0)$ with nothing added. Appendix A prints this program in full, verbatim from the proof.

The displayed tree is useful because it shows the *shape* of the program. The repository also exposes ordinary Lean functions obtained by evaluating the realizer at a certified ambient level and reading the relevant witness or tag. For the small examples this is short enough to print directly:

```
def fibonacci (n : Nat) : Nat :=
fstPT (fstPT (app1 fibPairedTheorem 4 n)) (defaultPT 4)

def pasTag (n k : Nat) : Nat :=
tag2 pasTotal 6 n k

def goodsteinStopTime (m : Nat) : Nat :=
witness1 goodsteinTheorem 11 m
```

These are not separate algorithms written after the proofs. Each is a reader for the realizer produced by the same generic extractor. The accompanying continuity theorem has the uniform form

$$\texttt{extract\_continuous}\ D\ \rho : \mathsf{Continuous2}\big(\mathsf{extract}(D, \rho, [], 1)\big),$$

and the collapse wrapper packages the same type-2 realizer as an element of CtQ2. Appendix B lists the exported functions for the case studies; Appendix A prints the larger Goodstein program tree that those one-line readers are reading from.

The surprise is the *range reached with so little*. One signature of twenty-one symbols. One extractor, with one small combinator per inference rule. One soundness proof, one case per rule. One continuity invariant, closed under the same handful of combinators. That fixed, modest apparatus — never enlarged across the whole tour — was enough to reach a structured puzzle solution, a decision procedure that draws the Sierpiński triangle, a strengthened-invariant iterator, a greatest-common-divisor calculator, a discrete intermediate-value search, and two theorems — Goodstein and Kirby–Paris — that genuinely need transfinite induction to ε_0, more than ordinary arithmetic can prove. Wildly different mathematics; the same few moving parts.

And the whole way down, the machine kept the accounting honest — not as a formality but as the thing that made the strongest claim sayable at all. The constructivity verdict of Act V is not an aspiration one hopes holds up under scrutiny; it is a fact the kernel will reprint on demand, for every program in the fragment: everything that runs rests on two inert axioms and nothing classical. A proof that needs more than Peano arithmetic became a program that needs less than a whiff of choice — and we know it needs less because the machine, asked directly, said so.

So the small system delivers what Act I promised. But notice what the last two acts have been quietly reporting. The level bookkeeping was extorted rather than designed; the pairing had to be hand-built or the audit would have failed; the Hydra's general theorem could not be *stated* inside the fragment at all. Three complaints, three coats on one body: the object language has a single sort, so every structured thing — a strategy, a colouring, an ordinal, a tree — has to be coded into a number before the fragment can mention it.

That the machine still works under those conditions is the achievement of Part I. What the coding *cost* is a question Part I has no way to answer, because it has nothing to compare against. Part II builds the comparison.

Part II
The same proof theory, native types

Act VIII — Give the proof its native types

Everything so far happened inside a deliberately minimal system: one sort, $\mathbb{N}$, and nothing else. That minimality was the point — it kept the audit short — but Act VI recorded its price as three separate "documented compromises," and they turn out to be one compromise wearing three coats: the general Hydra theorem lived outside the fragment because a strategy is a *function*; Sperner's coloring needed a special symbol because a coloring is a *function*; and five symbols entered by numeral graph because their recursions run through an *encoding* of trees into numbers. All three say: the object language cannot mention $\mathbb{N} \to \mathbb{N}$.

So we built the same machine again, over *Heyting arithmetic in all finite types* — the system modified realizability was invented for [7, 12]. This act reports what changed, what it cost, and what the extracted programs look like now. (The repository is `modified-realizability-haomega`; the first-order development stays in-tree as the reference implementation.) One word on the name: throughout this act, HA^ω denotes the object theory actually formalized — finite-type Heyting arithmetic *extended by* $\mathrm{TI}(\varepsilon_0)$ with its matching recursor, and by the imported case-study primitives below — an extension of HA^ω, not the bare textbook system.

The system, in four displays

Types are the finite types over $\mathbb{N}$, with a unit for erased certificates and two further bases:

$$\tau \;::=\; \mathbb{N} \mid \mathbf{1} \mid \mathsf{ord} \mid \mathsf{hyd} \mid \tau \to \tau \mid \tau \times \tau.$$

ord is the ordinal notations below ε_0 and hyd is the hydras — the two objects the first-order development had to *code* into $\mathbb{N}$. Both are inductive types here, comparison and surgery are pattern matching, and every certified fact about them is *inherited* from the coded version by pulling back along the evident encoding, which therefore survives only inside alignment proofs and appears in no computation at all. Act X reports what that buys, and it is measurable.

Terms are Gödel's System T — λ, application, pairing, the numerals, and a recursor rec *at every type* — plus a small stock of primitives (+, the ε_0-order $\prec$, and the case-study symbols good, hydra, ord, ..., each evaluated by the first-order development's proven, choice-free value layer — definable in System T in principle, imported as primitives in practice), plus one genuinely new construct:

$$\mathsf{tiRec} : \big(\mathbb{N} \to (\mathbb{N} \to \mathbf{1} \to \tau) \to \tau\big) \to \mathbb{N} \to \tau,$$

recursion along $\prec$. Rule and recursor arrive as a *pair*; that pairing is the subject of Act IX.

Formulas carry their realizer type as an *index*, φ : Formula $\Gamma\ \tau$, with the modified-realizability assignment built into the constructors:

$$\begin{array}{lcl}
s = t,\ \bot & : & \mathbf{1} \\
\varphi \wedge \psi & : & \tau_\varphi \times \tau_\psi \\
\varphi \vee \psi & : & \mathbb{N} \times (\tau_\varphi \times \tau_\psi) \\
\varphi \to \psi & : & \tau_\varphi \to \tau_\psi \\
\forall x^\sigma.\, \varphi & : & \sigma \to \tau_\varphi \\
\exists x^\sigma.\, \varphi & : & \sigma \times \tau_\varphi
\end{array}$$

There is no ambient level, no transport machinery, and — because substitution preserves the index *by construction* — not a single type-cast anywhere in the extractor. Equality is primitive at every type (revised from a type-0-only first design when Pascal's row-level equations proved unstatable without it), and *one* Leibniz rule replaces the fragment's twenty per-symbol congruence schemas.

Extraction now lands *in the object language*:

$$\mathsf{extract} : \mathrm{Deriv}\ \Delta\ \varphi \;\longrightarrow\; \mathrm{Tm}\ (\Delta{+}\Gamma)\ \tau_\varphi.$$

This is the structural upgrade over Act III, where the realizer was an opaque element of the pure-type hierarchy, printable only by walking the derivation instead. A realizer that is a *term* can be printed, normalized, emitted as Haskell, and reasoned about — the repository's three-view artifact (`EXTRACTED_HAOMEGA.md`, rewritten at every build) renders all thirteen realizers this way, and the displays below are its three renderings of one certified object each.

Thirteen extracted programs

theorem	statement (derived in the object theory)
Goodstein	$\forall m\,\exists t.\ \mathrm{good}(m,t)=0$
Kirby–Paris	$\forall h\,\exists t.\ \mathrm{hydra}(h,t)=0$
Hercules, ∀-strategy	$\forall h\,\forall f^{\mathbb{N}\to\mathbb{N}}\,\exists t.\ \mathrm{play}(f,h,t)=0$
Hercules, any head	$\forall h\,\forall f^{\mathbb{N}\to\mathbb{N}}\,\forall g^{\mathbb{N}\to\mathbb{N}}\,\exists t.\ \mathrm{playAt}(g,f,h,t)=0$
gcd, full specification	$\forall a\,\forall b\,\exists g.\ g \mid a \wedge g \mid b \wedge \forall d.(d \mid a \to d \mid b \to d \mid g)$
Pascal mod 2	$\forall n\,\forall k.\ \mathrm{pas}(n,k)=1 \vee \mathrm{pas}(n,k)=0$
Sperner 1D	$\forall n\,\forall c^{\mathbb{N}\to\mathbb{N}}.\ c\,0=0 \to c\,n=1 \to \exists k.\ k<n \wedge c\,k \neq c(k{+}1)$
Tower of Hanoi	$\forall n\,\exists \mathit{len}\,\exists \mathit{moves}^{\mathbb{N}\to\mathbb{N}}.\ \ldots$
Fibonacci	$\forall n\,\exists y.\ y=\mathrm{fib}(n)$
Fibonacci at type 2	$\forall f^{\mathbb{N}\to\mathbb{N}}\,\exists y.\ y=\mathrm{fib}(f(f\,0))$
Goodstein, typed ordinals	$\forall m\,\exists t.\ \mathrm{good}(m,t)=0$ (again, on ord)
Kirby–Paris, typed trees	$\forall h^{\mathsf{hyd}}\,\exists t.\ \mathrm{dead}(\mathrm{play}(h,t))=0$
Hercules any head, typed trees	$\forall h^{\mathsf{hyd}}\forall f\forall g\,\exists t.\ \mathrm{dead}(\mathrm{playAt}(g,f,h,t))=0$

Four of these the fragment could not even *state*: the two Hercules theorems and Sperner quantify over functions, and the type-2 Fibonacci is where the continuity theorem finally has content (Continuous2 of the extracted functional, with an explicit computable modulus $\max(f\,0, f(f\,0))+1$). The any-head Hercules closes what the audit had recorded as the remaining gap: a computable surgery move (hcutAt, the leftmost-head move with the head position as an argument) whose descent is the first-order `play_descends` read on codes, so quantifying the head strategy costs one primitive and one schema. One scope statement survives, verbatim from the audit: independence from PA is formalized nowhere.

Reading an extracted program, three ways

Fibonacci first, because it fits on a line. The raw extracted object — the realizer itself, with its erased certificate visible as ⋆:

```
(λx0. ((λx1. fst ((λx2. rec[⟨0, S 0⟩ | (λx3. (λx4. ((λx5. ⟨snd x5, (fst x5 + snd
x5)⟩) x4))) | x2]) x1)) x0), ⋆)
```

Collapsed — contentless parts elided, what remains is exactly the computational content — it is the same term minus the trailing ⋆-component: iterate $\langle a,b\rangle \mapsto \langle b, a{+}b\rangle$ from $\langle 0,1\rangle$, take the first projection. And as generated Haskell (see the certification note below):

```
(\x0 -> (((\x1 -> (fst ((\x2 -> (natRec (0, 1)
  (\x3 -> (\x4 -> ((\x5 -> ((snd x5), ((fst x5) + (snd x5)))) x4)))
  x2)) x1))) x0), ())
```

Now the one worth the whole apparatus: Goodstein's collapsed program, abridged to its skeleton with the bureaucracy elided (. . .), because the mathematics is *visible in it*:

```
(λm. (. . . tiRec[(λx. (λih. (. . .
  rec[⟨s, . . . ⟩                          — eqDec hits 0: return the stop time s
  |. . . (ih ord(S S S s, good(m, S s))) . . . ]   — else recurse at the descended ordinal
. . . )) ]  ord(S S 0, m) )                          — start at ord(2, m)
```

The transfinite recursor, the zero test on $\mathrm{good}(m,s)$, and the ε_0 descent $\mathrm{ord}(s{+}3, \mathrm{good}(m,s{+}1)) \prec \mathrm{ord}(s{+}2, \mathrm{good}(m,s))$ — the entire proof structure, readable in the program it became. The

extracted stopping-time function returns the published values $[0, 1, 3, 5]$ for $m = 0..3$, checked at every build together with the certificate $\mathrm{good}(m, \mathrm{stop}(m)) = 0$; the first-order extract, walled behind its ambient tower, could not evaluate past $m = 1$.

Act IX — Matched principles: a rule and its recursor

Adding $\mathrm{TI}(\varepsilon_0)$ to the object theory is not a free move, and it is worth being exact about why, because it is the one place where this development strengthens the logic rather than merely retyping it.

System T defines exactly the provably-total functions of PA. Goodstein's stopping time is not among them. So a proof rule strong enough to derive Goodstein's theorem cannot have its computational content expressed by the term language as it stands: extraction would reach the rule and have nothing to emit. There are two ways out, and only one of them is honest. The dishonest one is to let the *evaluator* do the work — to define the extracted program by an unbounded search, or by a recursion whose termination the metatheory knows but the object language does not. That buys running programs at the price of the thing being demonstrated, since the program is then no longer read off the proof.

The route taken here is the other one. The rule arrives with a term former:

$$\mathrm{TI}(\varepsilon_0) \quad \longleftrightarrow \quad \mathsf{tiRec} : \big(\mathbb{N} \to (\mathbb{N} \to \mathbf{1} \to \tau) \to \tau\big) \to \mathbb{N} \to \tau.$$

Read the type: at x, given the values at every y *together with a realizer of* $y \prec x$, produce the value at x. That is the rule's premise transcribed. And because the order premise is an equation, its realizer is contentless — the $\mathbf{1}$ argument — so the recursor cannot receive evidence that a recursive call is legal, and must re-decide $\prec$ itself and fall back when the test fails. The proof rule and the program construct are the same shape, and neither is doing work the other does not license.

We can state the principle generally, because the development applies it twice. When the object theory is strengthened by a proof principle whose content the term language cannot already express, the strengthening must come paired with a term former whose type is the principle's premise read as a type. Both instances here obey it: $\mathrm{TI}(\varepsilon_0)$ with tiRec over coded ordinals, and its twin over the structural notations of ord.

What does *not* obey it, and should be named plainly, is the second kind of extension: the case-study symbols. good, hydra, ord and the rest enter as primitives evaluated by the first-order development's verified value layers. They are definable in System T in principle; they are imported in practice, because defining hereditary base representation and tree surgery inside the object language is a project of its own and not this one. That is a convenience, not a principle, and it is why the name HA^{ω} is qualified every time it appears here: the object theory formalized is finite-type Heyting arithmetic extended by a matched pair *and* by a stock of verified primitives. A reader who objects that this is not the bare textbook system is right, and the more precise description is the more interesting one — a framework for finite-type realizability that is *extensible by matched proof/program principles*, with one worked extension of genuine proof-theoretic strength.

Act X — Three experiments

Acts I–VIII establish that the pipeline works, twice, over two object languages. What makes the second system worth building is that having two of them turns features of the setup into *variables*. Three can be moved one at a time, with the extraction mechanism held fixed, and in each case the extracted program registers the change.

Varying the proof: the program is the proof's fingerprint

Sperner 1D was proved here by the same forward-scan invariant as in Act IV: $P(m) := c\ m = 0 \vee \exists k{<}m.\ c\ k \neq c(k{+}1)$. Same theorem, same invariant — yet the extracted searches *differ measurably*. The first-order proof consults the invariant first and decides $c(m{+}1)$ second; its program returns the *first* crossing. The HA^{ω} proof decides $c(m{+}1) = 0$ first, so a fresh zero re-enters the left disjunct and discards any crossing already found; its program returns the *last*: on $[0, 1, 0, 1]$ the first-order extract answers 0, this one answers 2 — both certified crossings, at every input, by soundness. Nothing in Acts I–VII showed this cleanly: two proofs of one theorem, distinguishable by running their extracts. The extraction map is faithful to proof structure — which is the thesis of Act VII, here caught in the act.

Varying the representation

The first experiment varied the proof. The second holds the proof fixed — line for line — and varies how the objects it reasons about are *represented*. `GoodsteinTyped.lean` proves $\forall m\ \exists t.\ \mathrm{good}(m, t) = 0$ a second time, by transfinite induction on structural notations instead of codes — the same derivation line for line, with three symbols swapped (ord $\mapsto$ ord$^{\mathrm{o}}$, $\prec \mapsto \prec^{o}$, tiRec $\mapsto$ tiRec$^{\mathrm{o}}$). The extracted program contains *no coded ordinal*: zero occurrences of the coded ord, $\prec$ and tiRec, and the two programs are guarded equal at every input the build evaluates.

What that buys is measured, and it is a claim about *size*, not speed:

n	code (decimal digits)	tree (nodes)
10	3	13
1,000	1,412	67
100,000	1,782	83

No reproducible running-time difference between the two extracted programs was found, and none is claimed — Lean's naturals are GMP-backed, so arithmetic on a 1,782-digit code is cheap. What the typed layer removes is the encoding itself, and with it the decode and normal-form side conditions that the first-order ordinal proofs carry throughout.

For the *hydras*, though, the encoding was not merely tidy-up: it was the measured bottleneck. `HydraTyped.lean` gives trees the base type hyd, with the Kirby–Paris ordinal landing in ord, so a battle state and its measure are both structural; the descent is the first-order `play_descends` applied to trees with no encode/decode round trip, and the whole layer adds *one* rule where the coded version needs three. `HydraTree.lean` then re-proves the theorem, and its extract contains zero coded operations. The consequence is the sharpest measurement in this act: at the hydra whose published Kirby–Paris battle length is 37, the coded extract overflows the interpreter without taking a single step, and the tree extract returns 37. The wall was never the mathematics, the proof, or the type discipline — it was the pairing function.

With the general Hercules game re-proved on trees as well (`HerculesTree.lean`), the decoding programme is complete: every case study that once coded an object into $\mathbb{N}$ has a typed twin, and the coded modules survive only as the baselines their twins are measured against. That last port is also the cheapest, and instructively so — at the value level it cost *nothing*, because the any-head move and its descent were already functions and theorems about trees. What had been coded was never the mathematics; it was only the interface the one-sorted object language forced on it.

Varying the logical strength

The third axis is the one Part I already exercised, but it is only visible as a *variable* now that the other two have been moved. Take the same theorem shape — $\forall m\, \exists t.\ \ldots$ — and vary how much induction the proof is allowed. An ordinary induction extracts to rec, recursion along the successor. A proof that needs TI(ε_0) extracts to tiRec, recursion along $\prec$. The printed Goodstein skeleton above shows it directly: the outermost recursion is not over $\mathbb{N}$ but over the ordinal, and the induction hypothesis appears as the recursive call at the descended ordinal. Raising the proof-theoretic strength does not merely license more theorems; it changes the shape of the recursion the extractor emits, and the change is legible in the program text.

The comparison, tabulated

Setting the two systems side by side — same extraction mechanism, same theorems, different object language — gives the following. Every row is either a statement about what can be *said*, or a measurement.

	first-order (coded)	finite type (structural)
quantify a strategy	not statable	a variable $f^{\mathbb{N}\to\mathbb{N}}$
a colouring	the `look` primitive	a function $\mathbb{N} \to \mathbb{N}$
ordinal notations	naturals, via a pairing	base type ord
hydra trees	naturals, via a pairing	base type hyd
binders	two capture-avoidance devices	intrinsically typed; capture impossible
congruences	~20 per-symbol schemas	one Leibniz rule
the realizer	element of the pure-type tower	a term of the object language
ambient levels	up to 41 (gcd)	none
inference rules	76	45
gcd extract	certified; evaluates at no input	evaluates
Tower of Hanoi	reaches $n = 4$	reaches $n = 10$
Kirby–Paris, the 37-battle	overflows the evaluator	returns 37

The rows above the rule are about expressiveness and bookkeeping; the three below it are the ones that cost programs. And the honest reading of the division is that the first block *causes* the second: a strategy that cannot be quantified forces a metatheorem, an ordinal that must be coded forces arithmetic on thousand-digit integers, and the levels that exist only to move realizers between tiers are what stop `gcdWitness` from evaluating at any input at all.

Act XI — What the extracted programs satisfy

Act III proved that every extracted program is continuous, and the proof was real, but in the fragment it had nothing to bite on. Continuity constrains how a program may inspect an *infinite* input; every theorem the fragment can state quantifies over numbers, so every extract takes a number, and the constraint is satisfied vacuously. The machinery was pre-paid infrastructure.

With function variables it becomes a claim with content. The type-2 Fibonacci theorem $\forall f^{\mathbb{N}\to\mathbb{N}} \exists y.\ y = \mathrm{fib}(f(f\,0))$ extracts to a functional

$$\mathrm{hiProgram} : (\mathbb{N} \to \mathbb{N}) \to \mathbb{N},$$

whose input is an infinite object. The continuity theorem now says something one can check: the program consults only finitely much of f. For this program the finite part is explicit and computable —

$$\mathrm{hiModulus}(f) \;=\; \max\big(f\,0,\ f(f\,0)\big) + 1,$$

with the theorem that any g agreeing with f below that bound gives the same answer. It is also exhibited as an element of CtQ2, with a type-1 associate: the type-2 functional is represented by an ordinary function on numbers.

The sharper evidence that the theorem is not vacuous is negative. Consider

$$\mathrm{notAllZero}(\alpha) \;=\; \begin{cases} 0 & \text{if } \alpha\, n = 0 \text{ for every } n \\ 1 & \text{otherwise,} \end{cases}$$

a perfectly well-defined functional of the same type. Deciding it requires reading all of α, so it is not continuous — and the development proves the consequence: *no derivation extracts to it*. There is an inhabitant of the type on the far side of a line the logic cannot cross, and the line is drawn by a theorem rather than by inspection.

How the proof goes is worth one paragraph, because it is the same device that reappears in Act XII. The vendored Kleene–Kreisel development is indexed by pure-type *level* and has neither products nor general arrows, while realizers here live at arbitrary finite types. Rather than generalize that library, the proof carries an oracle-parameterized logical relation Tracked $\tau\ X$: continuity at base type is the vendored notion, and every other type gets a clause. No associates are ever constructed, so the pure-type tower's shape never has to be matched — and the induction runs over the constructors of the term language, one case each, where the first-order version needed a preservation lemma for each of some forty extraction combinators.

Act XII — From derivation to running code

The Haskell renderings carried a disclaimer for most of this development: *uncertified translation; the certified artifact is the term*. That disclaimer is now a theorem plus a short, explicit trusted base.

Certifying emitted code against GHC is not available to us — it would need a formal semantics of Haskell and a proof that GHC implements it. What is available is what a verified compiler actually proves: correctness against a formal semantics of the *target*. So the target fragment gets a syntax HsTm (untyped, since emission erases every distinction Ty makes), a big-step

evaluation relation with closures, and a translation hsOf. Agreement across a type erasure cannot be an equation, so it is a logical relation Rel $\tau\ x\ v$ — an equation at base types, closure under application at arrows. That is the same device the continuity theorem uses, put to a second purpose:

$$\text{hsOf\_correct}: \text{ related environments} \Rightarrow \exists v.\ \text{HsEval (hsOf } t)\ \rho\ v\ \wedge\ \text{Rel } \tau\ (t.\text{eval } e)\ v.$$

The emitter now *is* hsPrint ∘ hsOf, so the strings in the artifact come from the syntax tree the theorem is about; the regenerated artifact is byte-identical, which is the evidence that routing it through the certified translation changed nothing.

What remains trusted is worth stating precisely, because it is short: the hand-written prelude, whose meaning the model *defines* to be the corresponding Lean primitives; the printer, and GHC's parse of its output; and the seven programs that use TI(ε_0), for which the emitter has never produced running code — hsOf sends tiRec to a constructor with no evaluation rule, faithfully modelling the `error` the prelude emits. Six of the thirteen programs are covered, and which six is checked at every build.

Act XIII — The ledger, and what the two systems show

The audit, rerun

The footprints, from the machine, for the HA$^{\omega}$ library:

extract, the Fibonacci realizer	*no axioms*
continuity; every derivation; every running extract	`[propext, Quot.sound]`
soundness	`[propext, Classical.choice, Quot.sound]`

Choice enters soundness exactly where it did in Act V: through the value-layer *theorem proofs* discharging the ordinal-descent schemas, while the value-layer definitions — everything that runs — stay choice-free. The verdict of Part I therefore survives the upgrade unchanged: every extracted program that runs depends on two inert axioms and nothing classical, and the kernel will reprint that on demand.

What is not claimed

The ledger is more useful for what it refuses than for what it asserts, so here it is in one place. Independence from PA is formalized nowhere: Goodstein and Kirby–Paris are proved here as termination theorems, and that PA cannot prove them is not claimed. The typed ordinal layer is a result about representation *size*; no reproducible running-time improvement was found for the two Goodstein extracts, and none is asserted. The certified translation covers six of the thirteen programs — the seven that use TI(ε_0) are not covered, because the target has no transfinite recursor and the emitter has never produced running code for them. The case-study symbols are imported primitives, not System T definitions. And the coded modules retained as baselines are still coded: their hydra extracts still overflow the evaluator at codes 4 and 7, which is the measurement their typed twins are measured against, not a defect that was repaired in place.

None of these is a footnote added under pressure. Each is a claim the development at some point stated more strongly and then walked back when the machine, or a measurement, declined to support it.

The thesis

Part I answered the question the Prologue asked: a proof that needs more than Peano arithmetic becomes a program that needs less than a whiff of choice, and we know it does because the machine was asked. That is a result about *extraction* — the mechanism works, at strength, honestly.

Part II answers a question Part I could not pose. Holding the extraction mechanism fixed and moving one feature of the setup at a time, the extracted program registers each move. Change the proof and the program changes strategy: two derivations of Sperner's lemma, same theorem and same invariant, return different crossings. Change the representation and the program changes what it can finish: the same derivation, line for line, computes a 37-step hydra battle from trees that it cannot begin from codes. Change the logical strength and the recursion changes shape: rec becomes tiRec, and the ordinal descent is visible in the program text. Change the type and semantic constraints appear that were vacuous before: continuity acquires content exactly when a realizer first takes a function as input, and the functionals it excludes can be named.

Put together, the two parts say something narrower and more useful than "proofs have computational content." They say that the content is *sensitive* — to how the theorem was proved, to how its objects were represented, and to how much induction it was allowed — and that with the extraction mechanism held fixed and the axiom ledger machine-checked, those sensitivities can be measured rather than argued about. What had been coded was never the mathematics; it was only the interface the one-sorted object language forced on it, and the cost of that interface is now a number rather than an intuition.

That is how a proof becomes a program: not by a grand mechanism, but by a small one applied with discipline, checked by something that cannot be talked past — and, once there are two of them to compare, by a machine in which the consequences of proving things one way rather than another can be run.

Appendix A — Looking inside: the extracted program as a tree

Throughout the tour we have said that the extracted realizer *is* a program, and Act VII drew it as one (Figure 18). This appendix closes the loop: it shows the actual program, printed from the actual proof, by a command you can run.

There is an obstacle to seeing it that is worth stating, because overcoming it is the point. You cannot simply *evaluate* the extracted realizer and look at the result. For Goodstein the realizer is built by the transfinite recursion tiRecC, and evaluating it forces that recursion — which is exactly the astronomically long computation Table 2 could not finish. And even on the smallest example, full evaluation *over*-reduces: the realizer of $\exists s.\ \mathrm{good}(3, s){=}0$ computes to the single number 30 (it is Nat.pair 5 0, the witness paired with a contentless payload), so the 5 the proof supplied is hidden inside an opaque code.

The command `#realizer` avoids both problems by reading the *proof* structurally rather than

running the program: it walks the finite derivation, names the combinator at each rule, and collapses every proof-irrelevant sub-derivation (an equation, $\bot$) to a single "$\cdot$" leaf. It therefore terminates instantly on proofs whose programs cannot be evaluated, and it keeps the witness visible. On the smallest realizer it prints the pair the prose has described all along:

```
exI  ⟨witness = 5, ·⟩
   · ⟨contentless⟩  good(3,5) = 0
```

The witness 5 sits at the top, exactly where $\exists$-introduction is pairing; the body, being an equation, carries no runtime content and is one leaf. That is Reading 1 of Figure 18, now as literal output.

The same command, on Goodstein's *theorem*, prints the whole extracted stopping-step program — the one Figure 18 distilled and Figure 13 contrasted with a search. It is a 29-node tree, and it prints in well under a second, though the program it describes could never be run to completion at $m = 4$:

```
allI  (Λk)
   impE  (app)
      allE[0]  (inst)
         allE[ord(2,good(x2,0))]  (inst)
            tiEps0 / tiRecC  (recurse along ≺)
               allI  (Λk)
                  impI  (λ)
                     allI  (Λk)
                        impI  (λ)
                           orE  (case)
                              eqDec  ⟨decide good(x2,x3) = 0⟩
                              exI  ⟨witness = x3, ·⟩
                                 · ⟨contentless⟩  good(x2,x3) = 0
                              impE  (app)
                                 allE[ord(S(S(S(x3))),good(x2,S(x3)))]  (inst)
                                    allI  (Λk)
                                       impI  (λ)
                                          impE  (app)
                                             allE[S(x3)]  (inst)
                                                impE  (app)
                                                   allE[x5]  (inst)
                                                      wk  (weaken)
                                                         wk  (weaken)
                                                            wk  (weaken)
                                                               ax  (head hypothesis)
                                                   · ⟨contentless⟩  prec(x5,x1) = 1
                                             · ⟨contentless⟩  ord(S(S(S(x3))),good(x2,S(x3))) = x5
                                 · ⟨contentless⟩  ord(S(S(S(x3))),good(x2,S(x3))) = ord(S(S(S(x3))),good(x2,S(x3)))
      · ⟨contentless⟩  ord(2,good(x2,0)) = ord(2,good(x2,0))
```

Read it top to bottom and it is the program of Figure 18: bind the input (allI), name the initial ordinal, recurse along $\prec$ (tiEps0/tiRecC), and at each state case-split (orE) on a decided equality — return the current step (exI, the witness) if the value is 0, otherwise recurse on the next state, invoking the induction hypothesis (ax, the recursive call) and discharging the ordinal descent as a contentless leaf (prec(. . .)=1). This is the recursion whose termination Figure 12 certified and whose guarantee Figure 13 set against the naive loop. The Hydra's program is the same 29-node shape, for the reason Figure 14 gives.

Reproducing this. Both boxes are verbatim output. In the repository,

```
#realizer goodThreeExDeriv
#realizer goodsteinTheorem
```

prints the two trees above; the companion `#realizerCH` annotates each node with its logic rule and proposition, which is the three-column correspondence of Figure 18 generated mechanically. The command reads every derivation rule (it is total by construction, with no wildcard case), so it works unchanged on every theorem in the development.

For readers who want a less tree-like view, the repository also provides the generic command `#program`. It runs the same derivation-guided traversal but renders the extracted realizer as pseudocode: `ind` becomes `natRec`, `tiEps0` becomes `wellFoundedRecOnPrec`, `orE` becomes `case split`, `exI` becomes `return witness`, and equation proofs become `erase(...)`. This is a decompiled display of the extracted program, not a second certified artifact; the certified object is still `extract D`, with correctness from soundness and continuity from `extract_continuous`.

Appendix B — Catalogue of exported extracted programs

The previous appendix prints the realizer tree. This appendix records the ordinary functions exported by the case studies. Each row has the same form: a theorem in the fragment, a reader that applies the extracted realizer and pulls out a witness or tag, a correctness theorem obtained from soundness, and the uniform continuity/CtQwrapper.

Case	Extracted reader	Correctness theorem	Continuous program
Fibonacci	`fibonacci : Nat -> Nat` `fibNext : Nat -> Nat`	`fibonacci_spec` `fibNext_spec`	`fib_extract_continuous` `fibRealizesCtQ`
Pascal mod 2	`pasTag : Nat -> Nat -> Nat` `pasDecide : Nat -> Nat -> Nat`	`pasDecide_eq`	`pas_extract_continuous` `pasRealizesCtQ`
Tower of Hanoi	`hanoiSolution : Nat -> Nat -> Nat -> Nat -> Nat` `hanoiMoves : Nat -> List (Nat * Nat)`	move-count guards and the realized Hanoi specification	`hanoi_extract_continuous` `hanoiRealizesCtQ`
Euclid / gcd	`gcdWitness : Nat -> Nat -> Nat`	`gcdWitness_dvd`	`gcd_extract_continuous'` `gcdRealizesCtQ`
Sperner	`spernerWitness : Nat -> Nat -> Nat`	`spernerWitness_spec`	`sperner_extract_continuous'` `spernerRealizesCtQ`
Goodstein	`goodsteinStopTime : Nat -> Nat`	`goodsteinStopTime_spec`	`goodstein_extract_continuous` `goodsteinRealizesCtQ`
Hydra	`hydraBattleLength : Nat -> Nat`	`hydraBattleLength_spec`	`hydra_extract_continuous` `hydraRealizesCtQ`

The names in the last column all come from the same theorem:

$$\forall D : \mathsf{Deriv}(\emptyset, \varphi).\quad \mathsf{Continuous2}\big(\mathsf{extract}(D, \rho, [], 1)\big).$$

The first column differs from example to example because the theorem shapes differ. A theorem of shape $\forall x.\exists y.\varphi$ yields a witness reader like `goodsteinStopTime`; a theorem of shape $\forall x.\forall y.(\varphi \vee \psi)$ yields a tag reader like `pasTag`; a theorem whose conclusion packages two existentials, such as Fibonacci's strengthened invariant, yields two readers, `fibonacci` and `fibNext`. What changes is only how we read the realizer. The extractor, soundness theorem, continuity proof, and CtQcollapse are the same.

References

[1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[2] Gerhard Gentzen. Beweisbarkeit und unbeweisbarkeit von anfangsfällen der transfiniten induktion in der reinen zahlentheorie. *Mathematische Annalen*, 119:140–161, 1943. doi: 10.1007/BF01564760.

[3] R. L. Goodstein. On the restricted ordinal theorem. *The Journal of Symbolic Logic*, 9(2): 33–41, 1944. doi: 10.2307/2268019.

[4] William A. Howard. The formulae-as-types notion of construction. In J. P. Seldin and J. R. Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479–490. Academic Press, 1980. Written 1969.

[5] Laurie Kirby and Jeff Paris. Accessible independence results for Peano arithmetic. *Bulletin of the London Mathematical Society*, 14(4):285–293, 1982. doi: 10.1112/blms/14.4.285.

[6] Stephen C. Kleene. Countable functionals. In A. Heyting, editor, *Constructivity in Mathematics*, pages 81–100. North-Holland, 1959.

[7] Georg Kreisel. Interpretation of analysis by means of constructive functionals of finite types. In A. Heyting, editor, *Constructivity in Mathematics*, pages 101–128. North-Holland, 1959.

[8] John Longley and Dag Normann. *Higher-Order Computability*. Theory and Applications of Computability. Springer, 2015. doi: 10.1007/978-3-662-47992-6.

[9] Emanuel Sperner. Neuer beweis für die invarianz der dimensionszahl und des gebietes. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 6:265–272, 1928. doi: 10.1007/BF02940617.

[10] William W. Tait. Functionals defined by transfinite recursion. *The Journal of Symbolic Logic*, 30(2):155–174, 1965. doi: 10.2307/2270132.

[11] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

[12] A. S. Troelstra, editor. *Metamathematical Investigation of Intuitionistic Arithmetic and Analysis*, volume 344 of *Lecture Notes in Mathematics*. Springer, 1973. Modified realizability and its soundness: Ch. III, §3.4.

Appendix C — The realizer as a term

Appendix A had to work around an obstacle: in the fragment you cannot see the extracted program by evaluating it. Evaluation either over-reduces (the witness disappears into an

opaque code) or does not finish (the transfinite recursion is the astronomical computation itself). So `#realizer` reads the *proof* instead, walking the derivation to print a tree.

In Part II that obstacle is gone, and its absence is the structural point of Act VIII. The realizer is a term of the object language, so printing it is printing — no derivation walk, no separate viewer, and the same object that prints is the one that runs and the one the certified translation of Act XII compiles. Here is a complete extracted program, verbatim: the Pascal mod-2 decision procedure, whose output is the Sierpiński gasket drawn in Act IV. Contentless components are elided (the collapsed view); nothing else is abridged.

```
(λx0. rec[(λx1. rec[0 | (λx2. (λx3. S 0)) | fst ((λx2. rec[0 | (λx3. (λx4. S 0)) | x2]) x1)]) | (λx1. (λx2.
(λx3. rec[0 | (λx4. (λx5. rec[0 | (λx6. (λx7. S 0)) | fst ((λx6. rec[S 0 | (λx7. (λx8. rec[S 0 | (λx9. (λx10.
0)) | fst x8])) | x6]) ((((λx6. rec[(λx7. rec[S 0 | (λx8. (λx9. 0)) | x7]) | (λx7. (λx8. (λx9. rec[S 0 |
(λx10. (λx11. (((λx12. (λx13. ((λx14. rec[0 | (λx15. (λx16. ((λx17. rec[S 0 | (λx18. (λx19. 0)) | x17])
x16))) | x14]) (x12 + x13)))) (x8 x10)) (x8 S x10)))) | x9]))) | x6]) x1) x4) + (((λx6. rec[(λx7. rec[S 0 |
(λx8. (λx9. 0)) | x7]) | (λx7. (λx8. (λx9. rec[S 0 | (λx10. (λx11. (((λx12. (λx13. ((λx14. rec[0 | (λx15.
(λx16. ((λx17. rec[S 0 | (λx18. (λx19. 0)) | x17]) x16))) | x14]) (x12 + x13)))) (x8 x10)) (x8 S x10)))) |
x9]))) | x6]) x1) S x4)))]) | x3]))) | x0])
```

Two things are worth noticing about that block. It is *finite and complete* — there is no elision but the contentless one, and no appeal to a viewer. And it contains no operation that inspects an encoding: the recursion is rec over the row at a function type, the arithmetic is +, and the branching is the numeral test extraction emits for $\vee$-elimination.

That last property is what the de-coding programme of Act X bought, and it can be counted rather than asserted. Taking the three theorems that have both a coded and a structural proof, and counting occurrences of the coded operations — ord, $\prec$, tiRec, hcut, hydra, hord — in the structural extract:

structural extract	coded operations	structural operations
Goodstein on ord	0	$2 \times \mathrm{ord}^{\mathrm{o}}$, $1 \times \mathrm{tiRec}^{\mathrm{o}}$
Kirby–Paris on hyd	0	$7 \times \mathrm{cut}^{\mathrm{H}}$, $2 \times \mathrm{hord}^{\mathrm{H}}$, $1 \times \mathrm{tiRec}^{\mathrm{o}}$
Hercules on hyd	0	$7 \times \mathrm{cutAt}^{\mathrm{H}}$, $1 \times \mathrm{tiRec}^{\mathrm{o}}$

Zero is the whole claim. The encoding is not reduced in these programs; it is absent from them, surviving only inside the alignment proofs that let every certified fact about the coded version be inherited by the structural one.

Appendix D — Catalogue of the thirteen HA^{ω} programs

Appendix B catalogued the fragment's exports. This is the counterpart for Part II. Every row is one derivation; the realizer type is computed from the statement by the modified-realizability assignment of Act VIII, not chosen; and the last column records what the build checks at every compilation.

Theorem	Realizer type	Extracted program, and what is checked
Fibonacci	`N -> (N x 1)`	`fibExtracted`; values to $n = 15$, $n = 100$, and $n = 1000$ (209 digits)
Fibonacci at type 2	`(N -> N) -> (N x 1)`	`hiProgram`; continuous, modulus $\max(f0, f(f0))+1$, associate in CtQ2
Pascal mod 2	`N -> (N -> (N x (1 x 1)))`	`pasTag`/`pasDecide`; gasket rows 0–7 against the value layer
Tower of Hanoi	`N -> (N x ((N -> N) x (1 x 1)))`	`hanoiExtracted`; $n = 10$, all 1023 moves against a reference
gcd, full spec	`N -> (N -> (N x ((N x 1) x ((N x 1) x (N -> . . . )))))`	`gcdFull`; agrees with `Nat.gcd`
Goodstein	`N -> (N x 1)`	`goodsteinX`; stop times $[0, 1, 3, 5]$, each certified terminal
Kirby–Paris	`N -> (N x 1)`	`hydraX`; battle lengths $[0, 1, 3]$, each certified terminal
Sperner 1D	`N -> ((N -> N) -> (1 -> (1 -> (N x . . . ))))`	`spernerX`; a certified crossing — the *last* (Act X)
Hercules, ∀-strategy	`N -> ((N -> N) -> (N x 1))`	`herculesX`; agrees with `hydraX` at the fragment's own strategy
Hercules, any head	`N -> ((N -> N) -> ((N -> N) -> (N x 1)))`	`herculesAnyX`; agrees at the leftmost head strategy
Goodstein, typed ordinals	`N -> (N x 1)`	`goodsteinOX`; equal to `goodsteinX` at every evaluated input
Kirby–Paris, typed trees	`H -> (N x 1)`	`hydraHX`; codes 0–7, including 37 where the coded extract overflows
Hercules, typed trees	`H -> ((N -> N) -> ((N -> N) -> (N x 1)))`	`herculesTX`; 37 at the same hydra, under three strategy pairs

Six of the thirteen are additionally covered by the certified translation of Act XII — those that do not use TI(ε_0) — and which six is itself checked at every build.